M.TECH · CS-6418

Advanced Database Management System — Visual Study Book

ER model → Relational algebra & SQL → Normalization → Transactions & recovery → NoSQL.
Every topic with a diagram, a real-life use case, SQL and an exam tip.

Unit I · DBMS & ER model
Unit II · Algebra, Calculus, SQL
Unit III · FD & Normalization
Unit IV · Transactions & Recovery
Unit V · Graph, Spatial, Temporal, NoSQL

65 original diagrams · definitions to open every answer · Hinglish memory hooks · revision checklist

Table of Contents

How to use this book 3

DBMS Fundamentals & the ER Model 5

1.1 Advantages of the DBMS Approach 5
1.2 Views of Data & Data Independence 6
1.3 Schema and Sub-Schema 8
1.4 Data Models 8
1.5 Database Languages 8
1.6 Transaction Management (preview) 9
1.7 DBA, Users & Data Dictionary 9
1.8 Database Architectures 10
1.9 The ER (Entity-Relationship) Model 11
1.10 Domains, Relations & Types of Relations 17

Relational Algebra, Calculus & SQL 19

2.1 Structure of the Relational Model 19
2.2 Relational Algebra 20
2.3 Relational Calculus 22
2.4 Query-By-Example (QBE) 23
2.5 SQL 23

Functional Dependencies, Normalization & Integrity 29

3.1 Functional Dependencies (FD) 29
3.2 Introduction to Normalization 32
3.3 Normal Forms 34
3.4 Database Integrity 38

Transactions, Concurrency, Recovery & Query Processing 41

4.1 Transactions & ACID Properties 41
4.2 Concurrent Execution 43
4.3 Serializability 44
4.4 Concurrency Control 45
4.5 Deadlock 47
4.6 Failure Classification 48
4.7 Storage Types & Stable Storage 49
4.8 Data Access — Buffer Management 50
4.9 Log-Based Recovery 50
4.10 Query Processing & Optimization 52

Graph, Spatial, Temporal Databases & NoSQL 55

5.1 Graph Databases 55
5.2 Spatial Databases 56
5.3 Temporal Databases 58
5.4 NoSQL Databases 59

Quick comparison sheet 63

Exam-day revision checklist 65

Recommended resources 66

How to use this book

The 3-step exam answer formula

⭐ **Exam tip — Write every answer in this order**

1. **Definition** — one line (the blue 📘 boxes give it ready-made).
2. **Explain in 3–5 lines** — plain English with the **bold** keywords.
3. **Diagram, table or SQL** — draw the figure from the book; neat labelled diagrams earn marks.

Box guide

Box	What it gives you
📘 Definition	Opening line for the answer
🏭 Real-life use case	Where it is used — good for "applications" and viva
💡 Why it matters	The reason a feature exists
⭐ Exam tip	Marks-scoring points and exact statements to memorise
🧠 Yaad rakho	Short Hinglish memory hook
⚠️ Careful	Common mistakes and traps
❓ Likely exam questions	Past-paper style questions for the unit

Course outcomes

CO	Description
CO1	Strong foundation of query languages through relational algebra, calculus and QBE
CO2	Design conceptual, logical and physical database models using the ER model and normalization
CO3	SQL proficiency on simple and advanced features — concurrency, transactions, recovery in a multiuser environment
CO4	Skills for NoSQL-based database application development
CO5	Exposure to graph, spatial and temporal databases

Syllabus map

Unit	Hours	Focus	Main figures to practise
I	8	DBMS fundamentals, architecture, ER model, keys	Three-schema · ER symbols · ER → tables
II	8	Relational algebra, relational calculus, SQL	RA operators · join family · SQL joins
III	8	Functional dependencies, normalization, integrity	Closure · NF ladder · BCNF
IV	8	Transactions, concurrency control, recovery, query processing	Txn states · precedence graph · checkpoint
V	—	Graph, spatial, temporal databases and NoSQL	CAP · NoSQL types · valid vs transaction time

The big picture — how the five units fit

Stage	Unit	What happens
1. Model	I — ER model	Understand the real world and draw entities, relationships, keys; convert to tables
2. Query	II — Algebra & SQL	Ask questions of the data: σ π ⋈, SELECT, joins, views
3. Refine	III — Normalization	Use functional dependencies to remove redundancy and anomalies; add integrity rules
4. Run safely	IV — Transactions	Many users at once: ACID, locking, deadlock, crash recovery, fast query plans
5. Go beyond tables	V — Modern DBs	When data is connected, geographic, historical or huge: graph, spatial, temporal, NoSQL

🏭 Real-life use case — One online order, end to end

A shopper's order is designed with an **ER model** (Unit I), stored in normalised tables and queried with **SQL joins** (Units II–III), protected as a **transaction** so stock and payment stay consistent even if the server crashes (Unit IV), while recommendations come from a **graph database** and the cart sits in a **key-value store** (Unit V).

GFG links in this book open a GeeksforGeeks **search** for that topic (so they never break) — click the top result. Practice: geeksforgeeks.org/dbms · postgresql.org/docs · neo4j.com/docs · mongodb.com/docs

UNIT I · 8 HOURS

DBMS Fundamentals & the ER Model

DBMS vs files | Three-schema | Data independence | DDL/DML/DCL/TCL | DBA & dictionary | Architectures | ER model | Keys | EER | ER → tables

1.1 Advantages of the DBMS Approach

Definition

A **Database Management System (DBMS)** is software that enables the creation, storage, retrieval and management of data in a structured way, giving a controlled interface between users / applications and the underlying data.

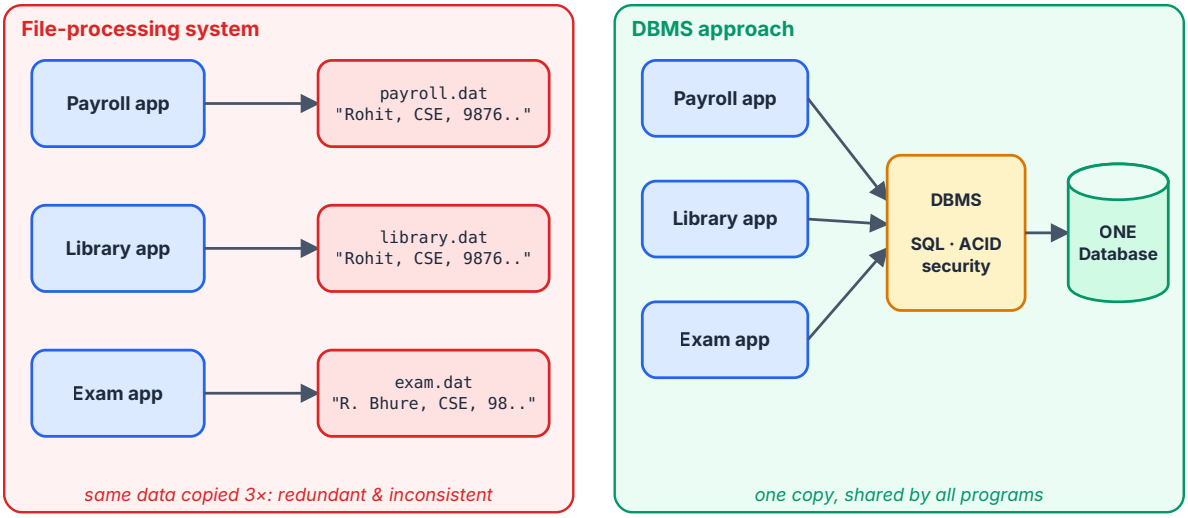


Fig. 1 — File-processing system (each app owns its files) vs a DBMS (one shared database)

Problem in file-processing systems	How the DBMS solves it
Redundancy & inconsistency — same data duplicated; an update in one copy is missed in others	Centralised storage — one copy, every application uses it
Difficulty in accessing data — new program for every new query	Query languages (SQL) give ad-hoc querying
Data isolation — scattered files in different formats	Unified data model brings data together logically
Integrity problems — constraints buried in application code	Constraints defined once, centrally, enforced by the DBMS
Atomicity problems — a crash mid-transfer leaves half-done data	Transactions with ACID properties (Unit IV)
Concurrent-access anomalies	Concurrency control — locking, timestamps (Unit IV)
Security problems — who may see / change what	Authorisation on relations, views, even attributes

🏭 **Real-life use case — Bank before and after**

Imagine a bank where the savings section and the loan section keep separate files. A customer who changes address at one counter keeps the old address in the other file — that is **redundancy + inconsistency**. With one DBMS, the address lives in one row and every app sees the same value.

⭐ **Exam tip**

List **all 7 points** briefly — "advantages of DBMS over file system" is one of the most repeated theory questions.

🧠 **Yaad rakho**

R-D-D-I-A-C-S: Redundancy, Difficulty accessing, Data isolation, Integrity, Atomicity, Concurrency, Security.

📖 GFG: Advantages of DBMS over File System

1.2 Views of Data & Data Independence

📘 **Definition**

The **three-schema (ANSI/SPARC) architecture** separates the user's view of the database from its physical storage through three levels of abstraction.

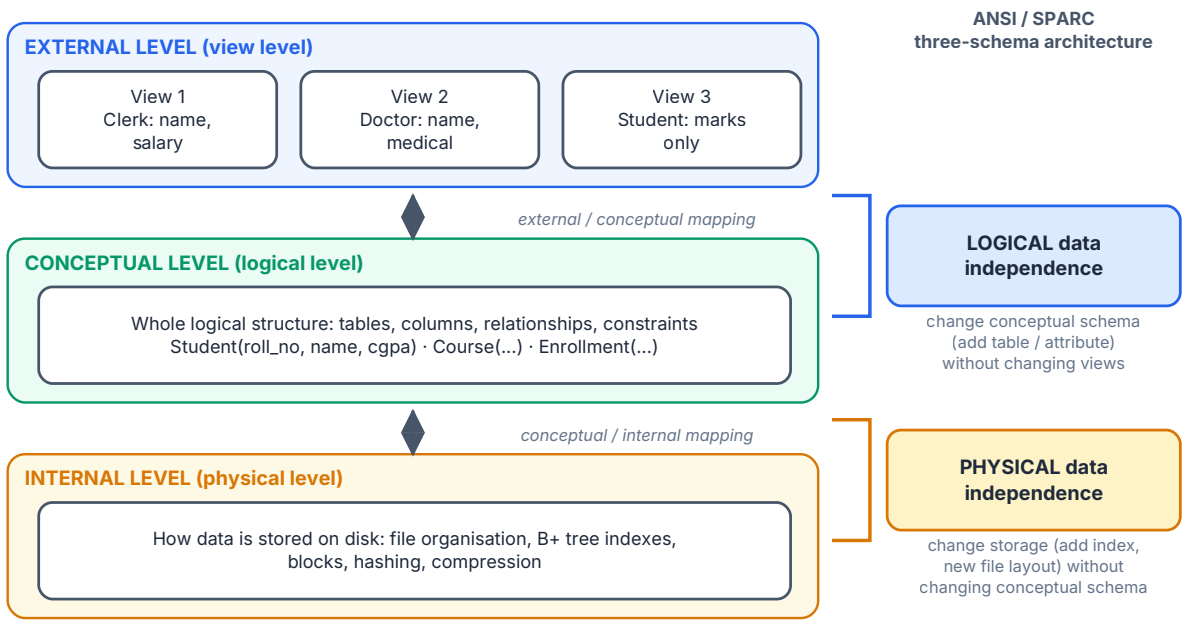


Fig. 2 — External, conceptual and internal levels with the two mappings between them

Level	What it describes
External / View	Highest level — only the part a user or application needs (a clerk sees salary columns, not medical records). Many views exist.
Conceptual / Logical	The whole logical structure — tables, relationships, constraints. Same for all users; the DBA works here.
Internal / Physical	How data is physically stored — file organisation, indexes, block sizes.

Data independence

📘 **Definition**

Data independence is the ability to change the schema at one level without having to change the schema at the next higher level.

Type	Meaning
Physical data independence	Change the internal schema (new file organisation, add an index) without touching the conceptual schema or application programs.
Logical data independence	Change the conceptual schema (add a table / attribute) without changing external views or programs that don't use the changed part.

⭐ **Exam tip**

Logical data independence is **harder** to achieve than physical, because applications are tied more tightly to the logical structure than to physical storage. Common 2-mark question.

🏭 **Real-life use case — Adding an index**

A DBA adds an index on `Student(roll_no)` to speed up searches. No SQL query or application code changes — only the internal level changed. That is physical data independence at work.

📖 GFG: Three-Schema Architecture & Data Independence

1.3 Schema and Sub-Schema

Term	Meaning
Schema	The overall logical design of the whole database — the blueprint. Changes rarely.
Instance	The actual data in the database at a moment in time. Changes constantly.
Sub-schema	A subset of the schema visible to one user / application (= the external level).

🧠 **Yaad rakho — house analogy**

Schema = **naqsha (blueprint)** of the house, fixed. Instance = **furniture** currently inside, keeps changing.

📖 GFG: Instance and Schema in DBMS

1.4 Data Models

📘 **Definition**

A **data model** is a collection of conceptual tools for describing data, data relationships, data semantics and data constraints.

Model	Idea
Relational	Tables of rows and columns — the most widely used today
Entity-Relationship	Entities, attributes, relationships — used for design (§1.9)
Object-based	Objects with attributes and methods (object-oriented DB)
Semi-structured	Flexible, self-describing: XML, JSON, NoSQL document model (Unit V)
Hierarchical & Network	Tree / graph structured — historical interest

📖 GFG: Data Models in DBMS

1.5 Database Languages

📘 **Definition**

Database languages are used to **define, manipulate and control** data in a DBMS.

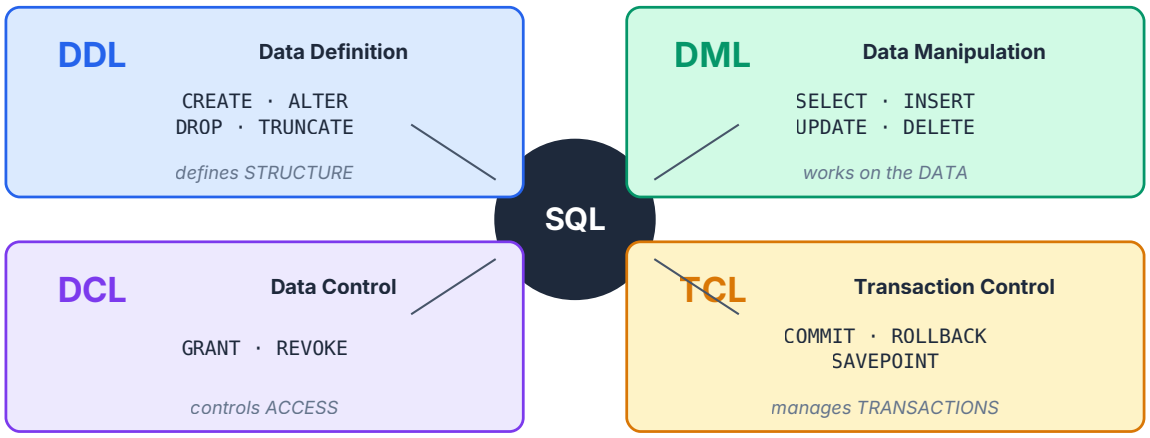


Fig. 3 — The four sub-languages of SQL and the commands in each

Lang.	Full form	Purpose	Commands
DDL	Data Definition Language	Defines / modifies the schema	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Manipulates the stored data	SELECT, INSERT, UPDATE, DELETE
DCL	Data Control Language	Controls access / permissions	GRANT, REVOKE
TCL	Transaction Control Language	Manages transactions	COMMIT, ROLLBACK, SAVEPOINT

DDL statements are compiled and stored in the **data dictionary**, which the DBMS consults before executing any DML statement to check that the request fits the schema.

📖 GFG: DDL, DML, DCL and TCL in SQL

1.6 Transaction Management (preview)

📘 Definition

A **transaction** is a logical unit of work made of one or more database operations that must run as an **all-or-nothing** unit to keep the database consistent.

The transaction manager guarantees the **ACID** properties — Atomicity, Consistency, Isolation, Durability — even if the system crashes midway or many transactions run together. Full detail is in Unit IV (§4.1).

1.7 DBA, Users & Data Dictionary

📘 Definition

The **Database Administrator (DBA)** is the person (or team) with central control over both the data and the programs that access it.

Functions of a DBA (write all 5)

1. **Schema definition** — creates the original schema using DDL.
2. **Storage structure & access-method definition** — decides physical storage and indexing.
3. **Schema & physical-organisation modification** — alters the schema as needs evolve.

4. **Granting authorisation** — who may access which data (DCL).
5. **Routine maintenance** — backups, disk space, performance monitoring, recovery from failures.

> 📘 **Definition — Data dictionary (system catalog)**
>
> A special set of tables maintained by the DBMS itself that stores **metadata — "data about data"**: table and column names / types, constraints, indexes, user permissions.

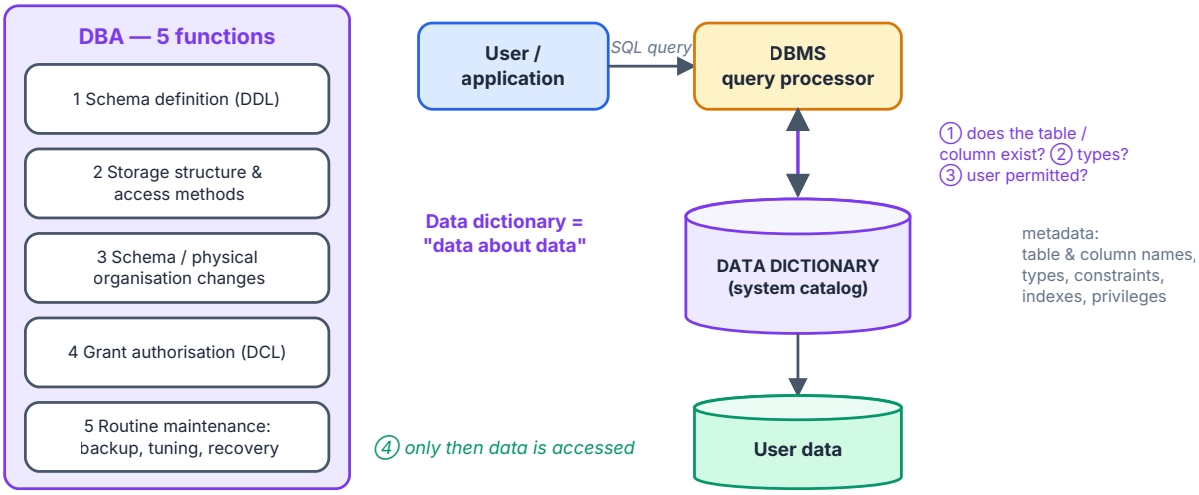


Fig. 4 — The data dictionary sits between every query and the stored data

> 💡 **Why it matters**
>
> Each query is first checked against the dictionary: do the tables and columns exist, are constraints respected? That is why it is called the **"brain" of the DBMS**.

📖 GFG: DBA & Data Dictionary

1.8 Database Architectures

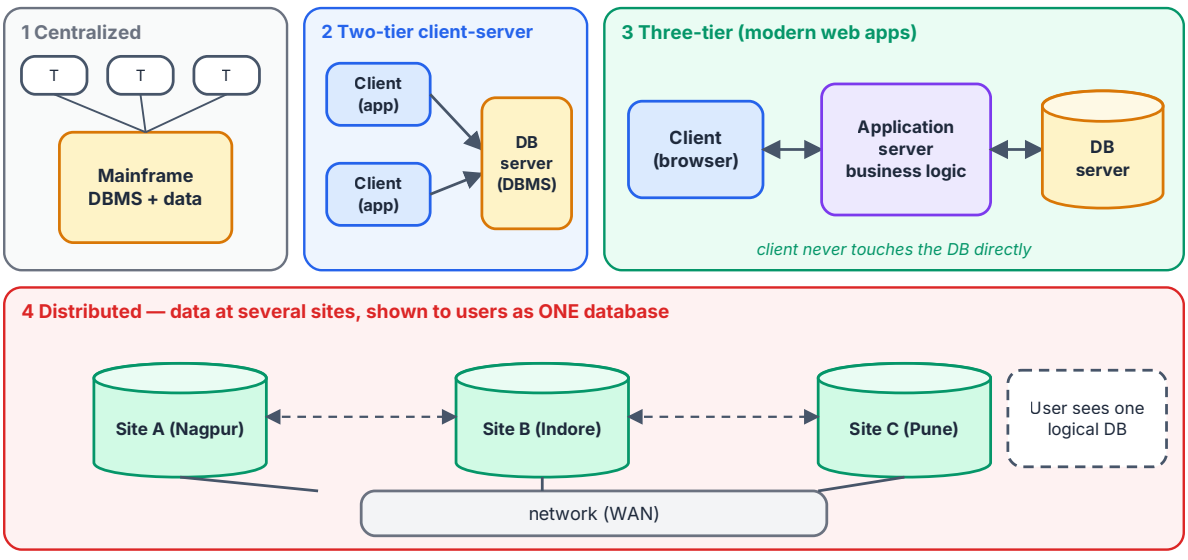


Fig. 5 — Centralized, client-server (two-tier / three-tier) and distributed architectures

Architecture	Key idea
Centralized	All data and DBMS on one machine; users connect through terminals
Client-server	Clients send SQL over a network; server processes and returns results. **Two-tier**: client ↔ DB server. **Three-tier**: client → application server (business logic) → DB server
Distributed	Data stored on multiple, possibly distant sites but presented as one database

⭐ **Exam tip — two-tier vs three-tier**

Three-tier adds an **application-server layer** that centralises business logic, improves **security** (clients never touch the DB directly) and makes the system easier to **scale**.

🏭 **Real-life use case — Where you see them**

A desktop billing app talking straight to MySQL is **two-tier**. A web shop (browser → Spring / Node server → PostgreSQL) is **three-tier**. A bank with branches in many cities, each holding its local data yet queryable as one, is **distributed**.

📖 GFG: DBMS Architecture

1.9 The ER (Entity-Relationship) Model

📘 **Definition**

The **ER model** is a high-level conceptual data model that describes the data, relationships and constraints of an application in a way non-technical stakeholders can follow, usually drawn as an **ER diagram**.

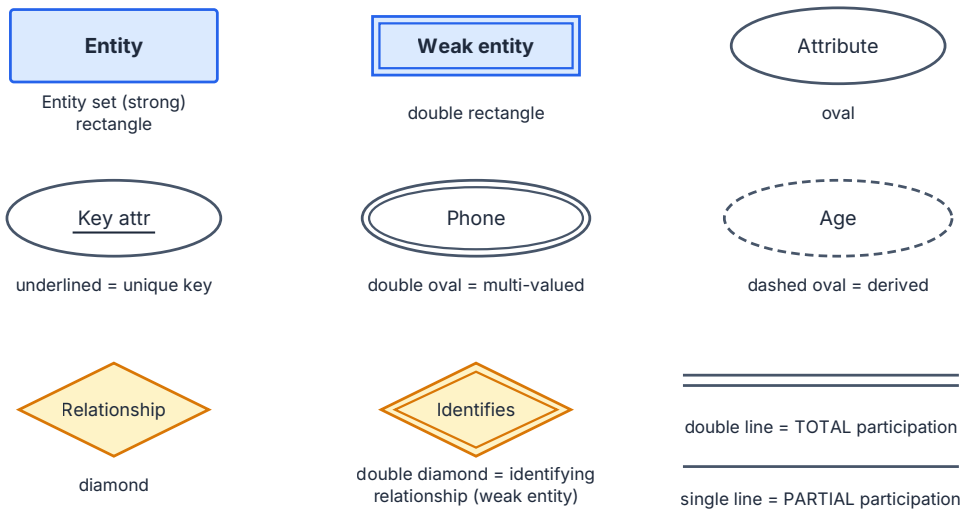
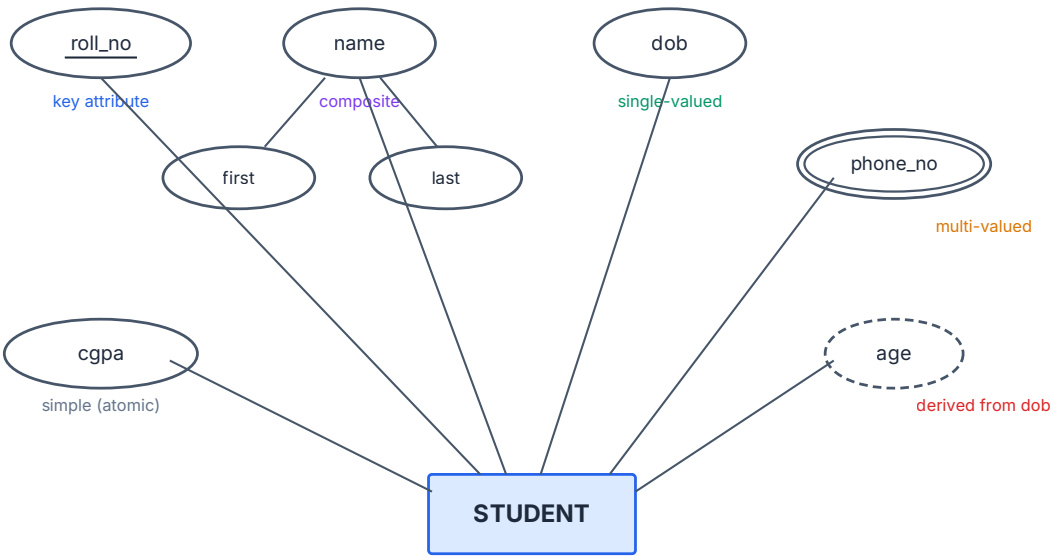


Fig. 6 — ER diagram symbols — one picture of every notation you must know

Concept	Definition	Symbol
Entity	Real-world object with independent existence, distinctly identifiable (a Student)	Rectangle
Entity set	Collection of entities of the same type (all Students)	Rectangle
Attribute	A property of an entity (name, roll_no)	Oval
Relationship	Association among entities (Student enrolls in Course)	Diamond
Relationship set	Collection of relationships of the same type	Diamond

Types of attributes



6 kinds of attribute: simple · composite · single-valued · multi-valued · derived · key

Fig. 7 — Six attribute types and how each is drawn

Attribute	Meaning / example
Simple (atomic)	Cannot be divided — roll_no
Composite	Divisible into parts — Name → (First, Last)
Single-valued	One value per entity — date_of_birth
Multi-valued *(double oval)*	Several values — phone_numbers
Derived *(dashed oval)*	Computed from others — age from date_of_birth
Key *(underlined)*	Uniquely identifies each entity

Design issues in ER modelling

1. **Entity set vs attribute** — is Address its own entity or just an attribute? Depends on whether it needs identity or relationships.
2. **Entity set vs relationship set** — is "Enrollment" an entity or a relationship between Student and Course?
3. **Binary vs n-ary** — most are binary, but some truly need ternary (Supplier–Part–Project).
4. **Placement of relationship attributes** — `date_of_enrollment` belongs to the *enrolls in* relationship, not to Student or Course alone.

Mapping cardinalities and participation

Definition

Mapping cardinality expresses how many entities of one set can be associated with entities of another set through a relationship.

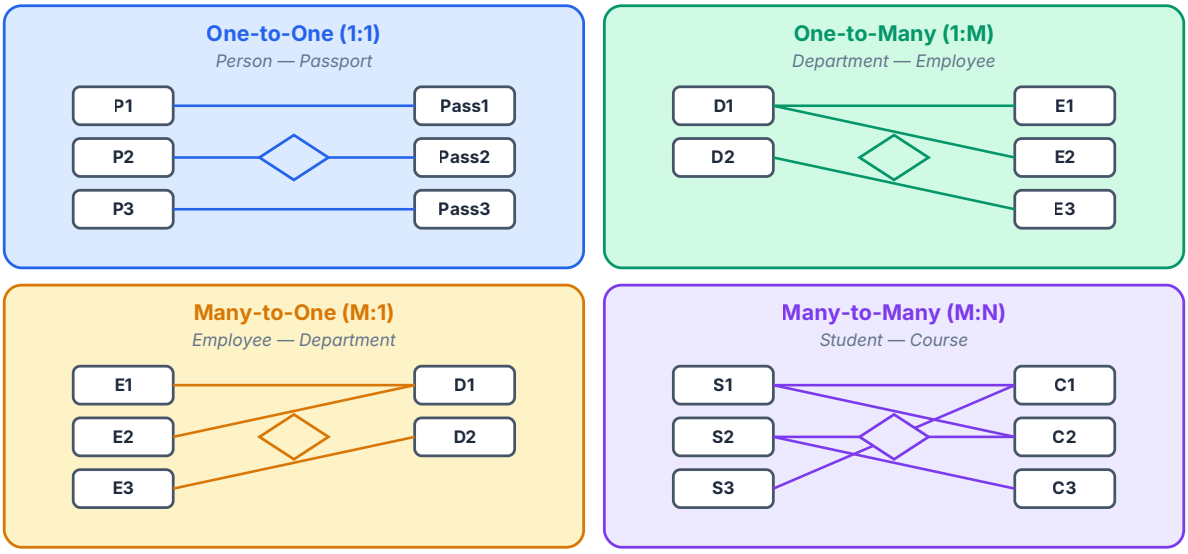


Fig. 8 — 1:1, 1:M and M:N with real examples, plus total vs partial participation

Type	Example
One-to-One (1:1)	A Person has exactly one Passport and vice versa
One-to-Many (1:M)	One Department has many Employees; each Employee in one Department
Many-to-One (M:1)	Reverse view of 1:M
Many-to-Many (M:N)	A Student takes many Courses; a Course has many Students

Total participation (double line): every entity must take part in the relationship, e.g. every Loan belongs to some Customer. **Partial participation** (single line): some entities may not take part.

Keys

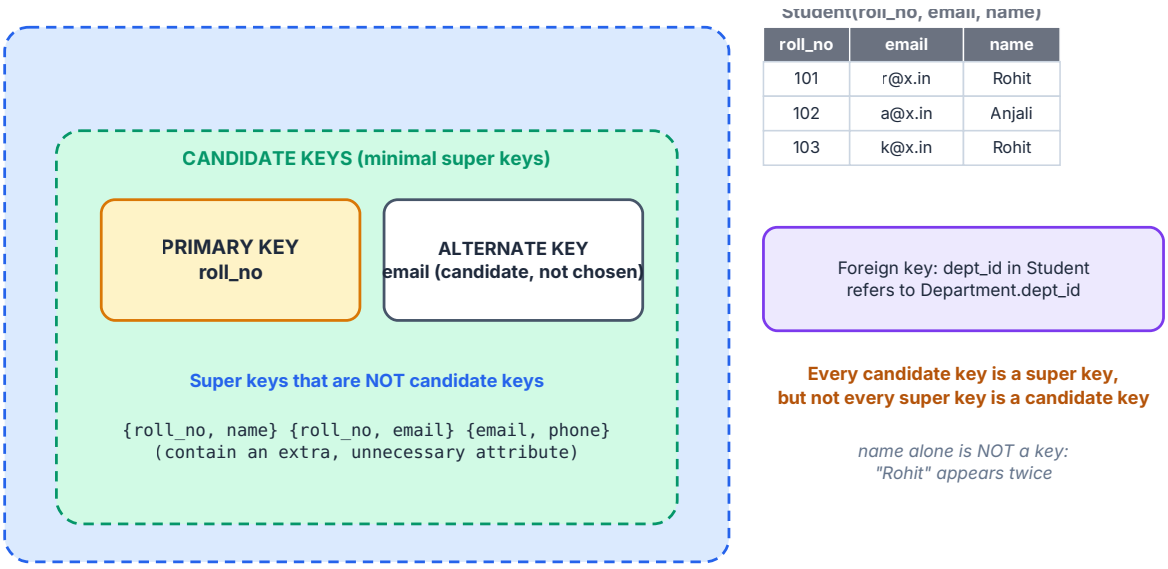


Fig. 9 — Super key ⊇ candidate key ⊇ primary key — the nesting that examiners test

Key	Definition
Super key	Any set of one or more attributes that together uniquely identify an entity
Candidate key	A **minimal** super key — no proper subset is also a super key
Primary key	The candidate key chosen by the designer as the main identifier
Alternate key	Candidate keys not chosen as primary key
Foreign key	Attribute(s) in one relation referring to the primary key of another (or the same) relation

⭐ **Exam tip**

Every candidate key is a super key, but not every super key is a candidate key — justify with the minimality condition. Write all five definitions from memory; they repeat in Units I, II and III.

🏭 **Real-life use case — Student table**

`Student(roll_no, email, name)` : {roll_no} and {email} are each unique → both candidate keys. Choose `roll_no` as **primary**; `email` becomes the **alternate** key; `(roll_no, name)` is a super key but **not** candidate (name is redundant).

Weak vs strong entity sets

📘 **Definition**

A **strong** entity set has its own primary key. A **weak** entity set lacks sufficient attributes for a primary key; it depends on an owner (identifying) strong entity, and is identified by **partial key (discriminator) + owner's primary key**.

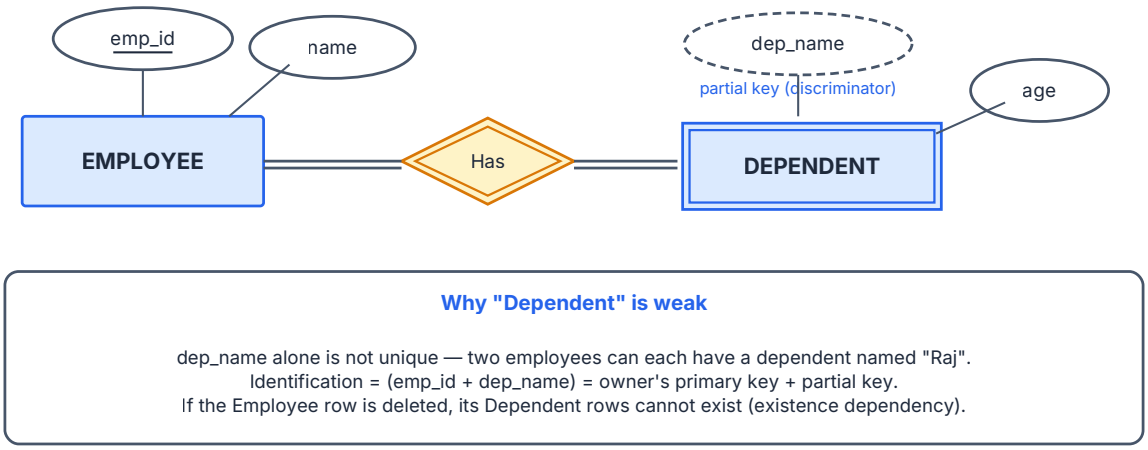


Fig. 10 — Employee (strong) and Dependent (weak) — double rectangle, double diamond, dashed underline

Example: two employees may each have a dependent named "Raj", so `dependent_name` alone is not unique; the weak entity is identified as `(emp_id, dependent_name)`.

Specialization, generalization, aggregation, inheritance

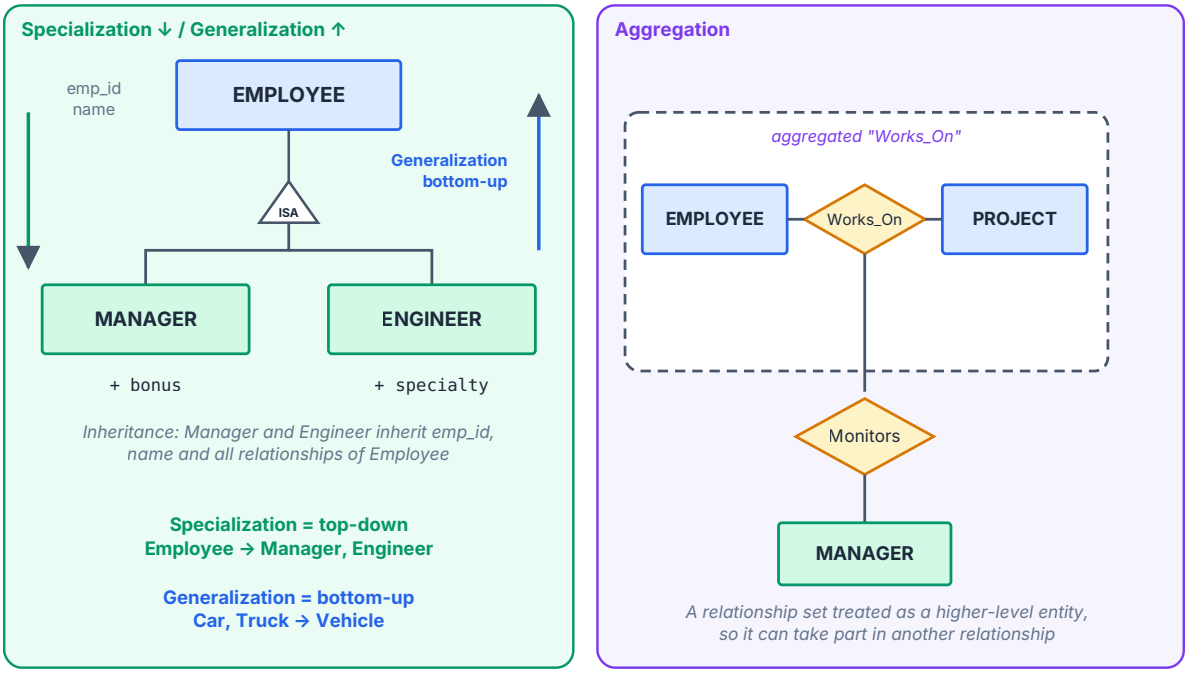


Fig. 11 — The four "advanced ER" ideas in one figure

Concept	Meaning & example
Specialization	Top-down: split a general entity set into sub-sets. Employee → Manager, Engineer
Generalization	Bottom-up: merge entity sets with common features. Car, Truck → Vehicle
Aggregation	A relationship set treated as a higher-level entity so it can join another relationship. (Employee, Project) *Works_On* is aggregated to take part in *Monitors* with Manager
Inheritance	Lower-level sets inherit all attributes and relationships of the higher-level set, like OOP

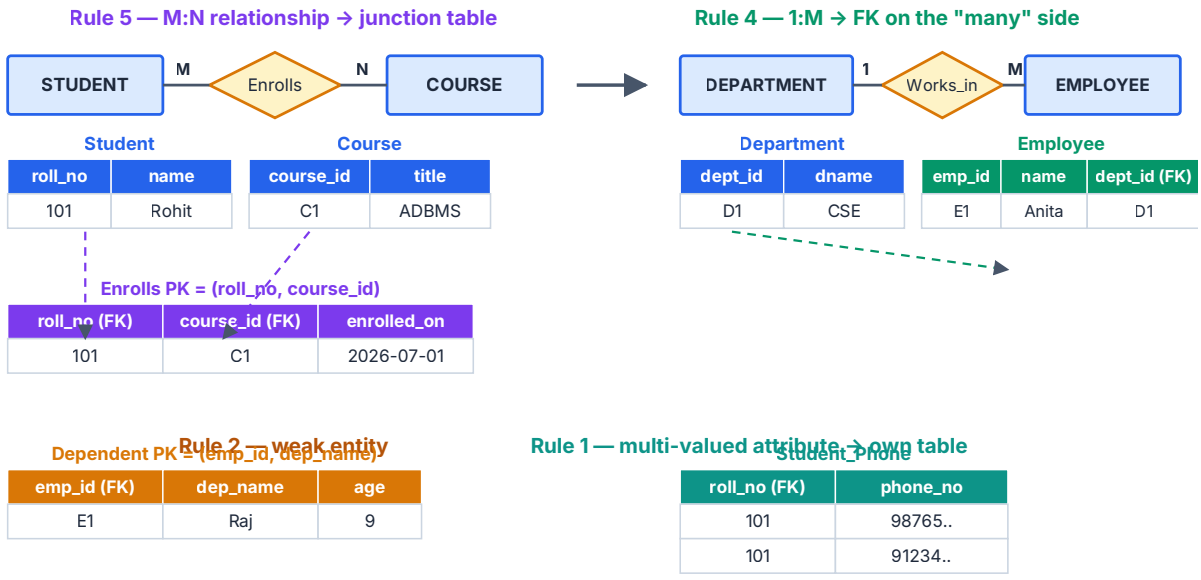
> ⚠️ **Careful**
>
> Specialization and generalization are **inverse processes**: specialization = general → specific (top-down); generalization = specific → general (bottom-up).

Steps to design an ER schema

1. Identify entity sets.
2. Identify relationships between them.
3. Identify and classify attributes (simple / composite, single / multi-valued, derived).
4. Decide cardinality ratios and participation constraints.
5. Choose primary keys; handle weak entities separately.
6. Apply specialization / generalization / aggregation where needed.
7. Draw the full diagram and review it against requirements.

Reduction of an ER schema to tables (memorise all 6 rules)

> 📘 **Definition**
>
> Converting an ER diagram into a set of relational tables — a mandatory step before implementing in a relational DBMS.



Rule 3: 1:1 → FK on either side (prefer the total-participation side) · Rule 6: specialization → single table with type column, or one table per subclass

Fig. 12 — The six reduction rules shown on one small schema

#	ER element	Becomes
1	Strong entity	Table with its simple / derived attributes; PK = entity key. Composite → its simple parts. Multi-valued → separate table (entity PK + the attribute)
2	Weak entity	Table with own attributes + owner's PK as FK; PK = owner's PK + partial key
3	1:1	PK of either side as FK in the other (prefer the total-participation side)
4	1:M	PK of the "1" side becomes FK in the "many" side table
5	M:N	New junction table: both PKs as FKs (together the composite PK) + relationship attributes
6	Specialization / generalization	(a) one table + type column; (b) a table per subclass with only its own attributes + inherited PK (total specialization); (c) a table per subclass duplicating all inherited attributes

```sql
-- M:N Student–Course  =>  junction table (rule 5)
CREATE TABLE Enrolls (
  roll_no   INT REFERENCES Student(roll_no),
  course_id INT REFERENCES Course(course_id),
  enroll_date DATE,                      -- relationship attribute
  PRIMARY KEY (roll_no, course_id)
);
```

🧠 **Yaad rakho**

1:M → FK on the Many side. M:N → naya table. Weak entity ka PK = owner PK + partial key.

📖 GFG: ER diagram to tables · Generalization, Specialization, Aggregation

1.10 Domains, Relations & Types of Relations

📘 **Definition — Domain**

The set of allowable (legal) values an attribute can take, e.g. age ∈ integers 0–150.

📘 **Definition — Relation**

A mathematical table — formally a subset of the Cartesian product of a list of domains — representing entities and relationships as rows (tuples) and columns (attributes).

Properties of a relation

- Each cell holds a **single atomic value** (the 1NF requirement).
- Each column has a distinct name; all values in a column come from the same domain.
- Order of rows and columns is immaterial (a relation is a **set** of tuples).
- No two tuples are identical.

Kind	Meaning
Base relation	Physically stored table created by `CREATE TABLE`
Derived relation / View	Virtual relation computed on demand from base relations (`CREATE VIEW`); not separately stored

❓ Likely exam questions — Unit I

- Advantages of DBMS over file-processing systems.
- Explain the three-schema architecture. Differentiate physical vs logical data independence.
- Schema vs instance vs sub-schema.
- DDL, DML, DCL, TCL with examples.
- Functions of a DBA. What is a data dictionary?
- Two-tier vs three-tier architecture.
- Types of attributes with ER symbols.
- Mapping cardinalities and participation constraints.
- Super, candidate, primary, alternate and foreign keys.
- Weak vs strong entity set (with example).
- Specialization, generalization, aggregation.
- Rules for reducing ER diagram to tables; draw ER for a library / hospital / college.

UNIT II · 8 HOURS

Relational Algebra, Calculus & SQL

Relational terms | Relational algebra | Joins & division | TRC / DRC | QBE | SQL SELECT | Aggregates & NULL | Subqueries | Views | DML / DDL

2.1 Structure of the Relational Model

Definition

The **relational model** represents data and relationships as a collection of **relations (tables)**, each made of tuples (rows) and attributes (columns), with a relational schema describing name + attributes + domains.

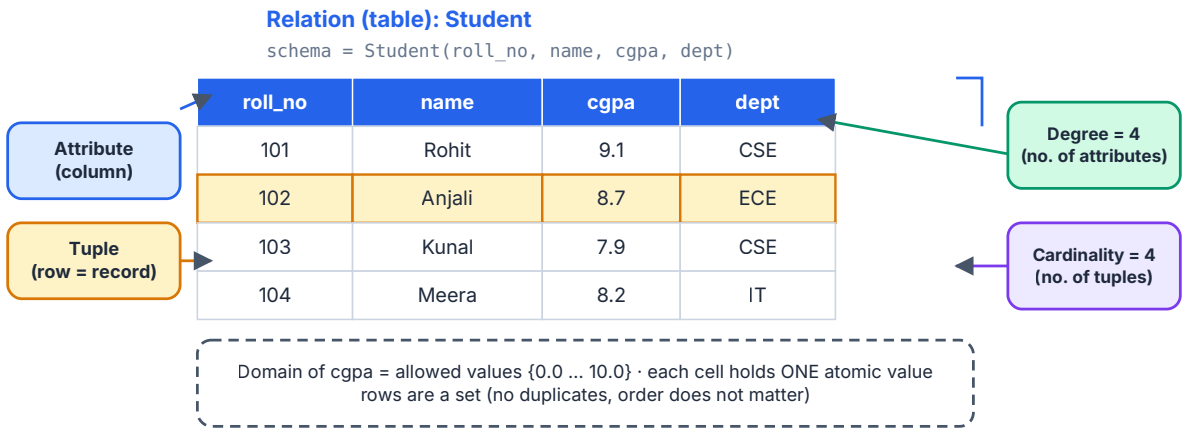


Fig. 13 — One table, every term labelled — tuple, attribute, degree, cardinality, schema, instance

Term	Meaning
Tuple	A single row — one record
Attribute	A column
Degree	Number of attributes (columns)
Cardinality	Number of tuples (rows)
Relation schema	Relation name + attribute list, e.g. `Student(roll_no, name, cgpa)`
Relation instance	The actual set of tuples at a given time

Yaad rakho

Degree = Direction of columns (vertical count). Cardinality = Count of rows. Dono ulte mat karna!

2.2 Relational Algebra

Definition

Relational algebra is a **procedural** query language: a set of operations that take one or two relations and produce a new relation — it says *how* to get the result, step by step.

Fundamental operations

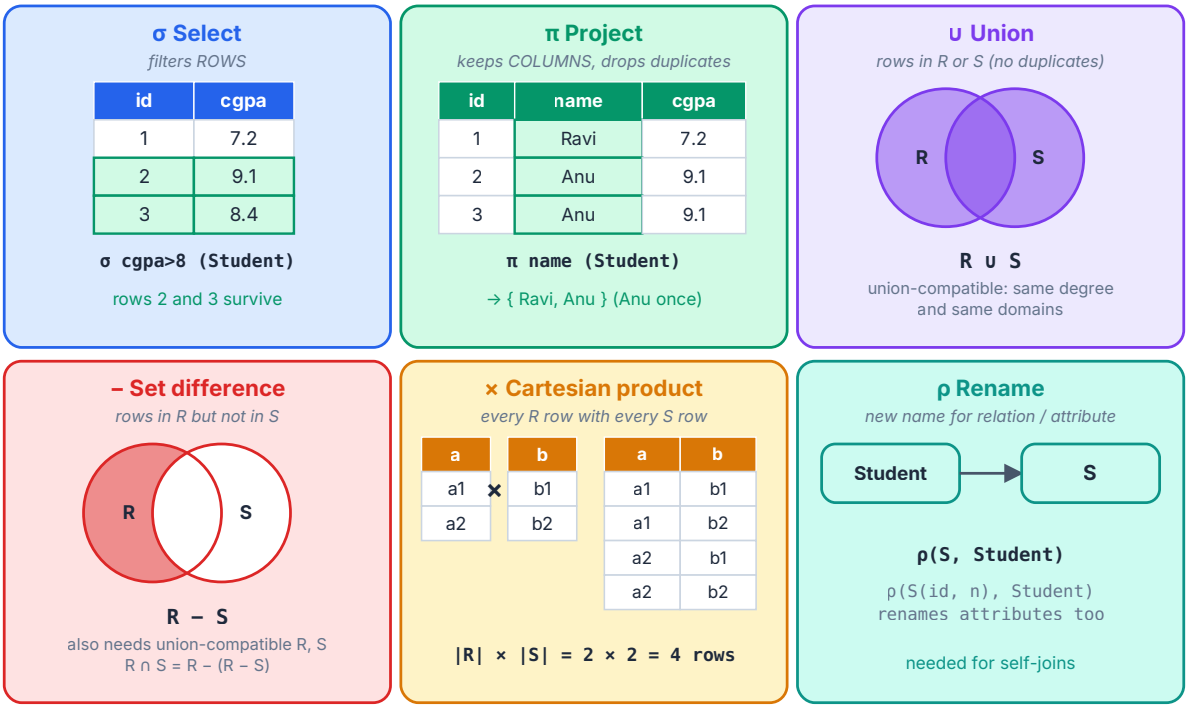


Fig. 14 — The six basic operators — what each does to a small table

Operation	Symbol	Meaning	Example
Select	σ	Picks rows satisfying a predicate	$\sigma_{cgpa>8}(Student)$
Project	π	Picks columns; removes duplicates	$\pi_{name,cgpa}(Student)$
Union	∪	Tuples in either relation (no duplicates)	$\pi_{name}(Student) \cup \pi_{name}(Alumni)$
Set difference	−	In first, not in second	R − S
Cartesian product	×	Every tuple of one with every tuple of the other	Student × Course
Rename	ρ	Renames relation / attributes (self-joins)	ρ(S, Student)

⚠ Careful — Union-compatibility

Two relations are union-compatible only if they have (a) the **same number of attributes** and (b) the **same domain** for each corresponding attribute. Needed for union, intersection and set difference.

Extended (derived) operations

Operation	Symbol	Meaning
Intersection	$\cap$	Tuples in both — derivable as R − (R − S)
Theta join	$\bowtie_\theta$	Tuples of R × S satisfying condition θ ≡ $\sigma_\theta(R \times S)$
Equijoin	$\bowtie_=$	Theta join using only equality
Natural join	$\bowtie$	Equijoin on all same-named attributes; duplicate column removed
Division	÷	"Find X related to **all** Y" — students enrolled in all courses
Outer join	⟕ ⟖ ⟗	Keeps unmatched tuples, fills NULLs (left / right / full)

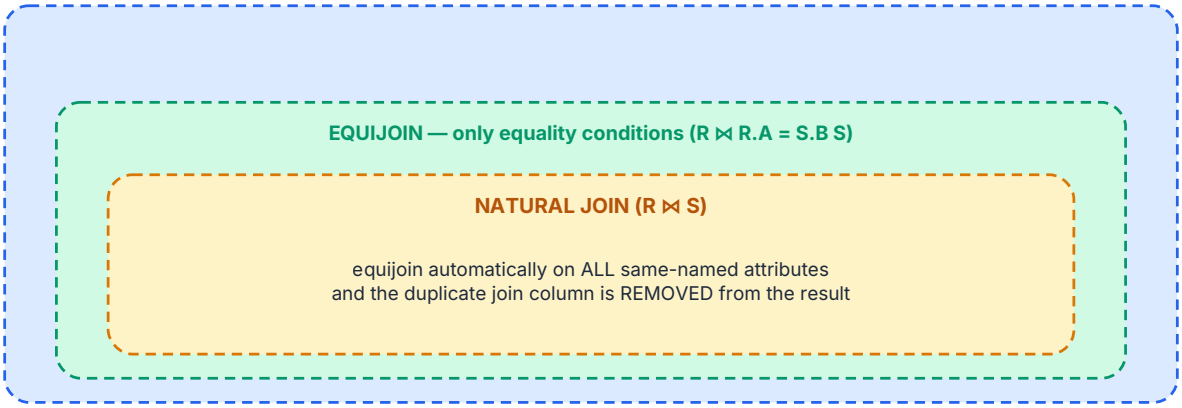


Fig. 15 — Theta → equijoin → natural join: each is a special case of the previous

- **Theta join** — most general; any comparison (<, >, =...).
- **Equijoin** — theta join restricted to **=**.
- **Natural join** — automatic equijoin on same-named attributes and it **drops the duplicate column**.

"Find students who enrolled in ALL courses"

Enrollment

student	course
S1	DB
S1	OS
S2	DB
S3	DB
S3	OS

÷

Course

course
DB
OS

=

Result

student
S1
S3

S2 is missing OS, so S2 is dropped

Division ÷ keeps X only if it is paired with EVERY row of the divisor

Fig. 16 — Division — which students took every course in the list?

🏭 Real-life use case — Division in real life

"Which suppliers supply **all** parts we need?", "Which employees worked on **every** project?" — any *for-all* question maps to division.

Modification in relational algebra

Operation	Expression
Insert	$r \leftarrow r \cup E$
Delete	$r \leftarrow r - E$
Update	$r \leftarrow \pi_{\text{modified attributes}}(\sigma_{\text{condition}}(r))$ — generalised projection over selected tuples

Each modification builds a **new relation** that replaces the old value of r — cleaner than SQL's "change in place" thinking.

> ⭐ **Exam tip**
>
> Natural join vs theta join vs equijoin is a favourite comparison. Always say: **equijoin ⊂ theta join; natural join = equijoin on common attributes without duplicate column**.

📖 GFG: Relational Algebra in DBMS

2.3 Relational Calculus

> 📘 **Definition**
>
> **Relational calculus** is a **non-procedural (declarative)** query language: you state *what* result you want using logical predicates, not how to compute it.

Form	Idea & example
Tuple RC (TRC)	`{ t | P(t) }` — variables range over **tuples**. Example: `{ t | t ∈ Student ∧ t.cgpa > 8.0 }`
Domain RC (DRC)	Variables range over **attribute values**: `{ <x1,…,xn> | P(x1,…,xn) }`. Example: `{ <n,c> | ∃r (r ∈ Student ∧ r.name = n ∧ r.cgpa = c ∧ c > 8.0) }`

Relational algebra vs relational calculus

Relational algebra	Relational calculus
Procedural — sequence of operations	Non-procedural — only *what* is required
Operators σ π ∪ × ⋈	Predicate / first-order logic formulas
Operational, closer to execution	Formal specification, closer to how SQL is designed

Equivalent in expressive power — every algebra expression ↔ a calculus expression. This equivalence is called **relational completeness**.

> 🧠 **Yaad rakho**
>
> Algebra = **"kaise"** (how). Calculus = **"kya"** (what).

2.4 Query-By-Example (QBE)

Definition

QBE is a visual, non-procedural query language: the user fills example values and conditions into on-screen **table skeletons** instead of typing query syntax.

Skeleton table shown on screen

Student	roll_no	name	cgpa
		P.	> 8
	_x		

```
P. = print this column

> 8 = condition on cgpa

_x = example variable (link
 values across tables)
```

Equivalent SQL: `SELECT name FROM Student WHERE cgpa > 8;`

Fig. 17 — A QBE skeleton for "names of students with cgpa above 8"

- Constants specify exact matches.
- Example variables are prefixed with an **underscore** (`_x`) to link values across tables.
- **P.** means "print this column" in the output.
- The system translates the picture into a formal query internally.

2.5 SQL

Definition

SQL (Structured Query Language) is the standard language for defining, manipulating and querying relational databases — DDL, DML, DCL and TCL in one language.

Basic SELECT structure and execution order

```sql
SELECT   column1, column2, ...
FROM     table1, table2, ...
WHERE    condition
GROUP BY column
HAVING   group_condition
ORDER BY column;
```

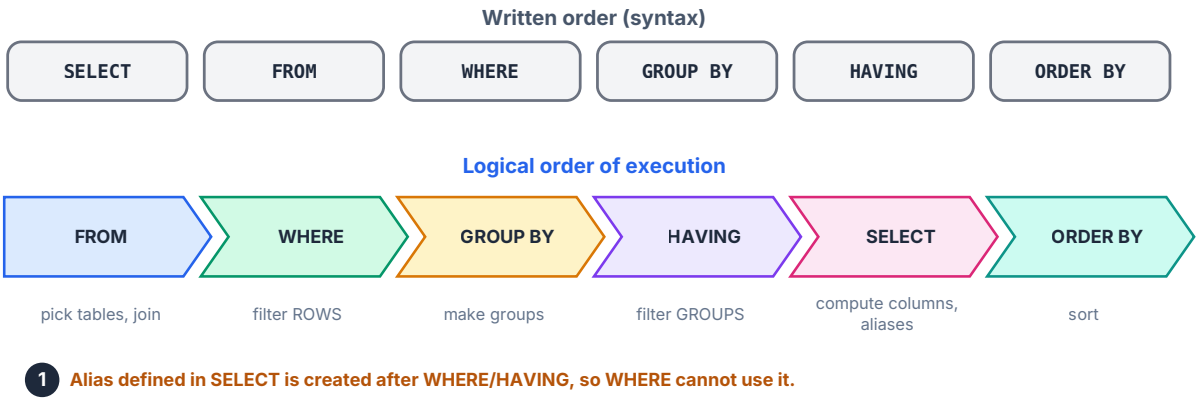


Fig. 18 — Written order vs logical execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

> ⚠ **Careful — Why aliases fail in WHERE**
>
> SELECT is written first but evaluated **after** FROM, WHERE, GROUP BY and HAVING — so you cannot use a SELECT alias inside WHERE.

Set operations

```sql
SELECT name FROM Student  UNION      SELECT name FROM Alumni;
SELECT name FROM Student  INTERSECT  SELECT name FROM Alumni;
SELECT name FROM Student  EXCEPT     SELECT name FROM Alumni;   -- MINUS in Oracle
```

UNION removes duplicates; **UNION ALL** keeps them and is faster (no duplicate elimination).

Aggregate functions, WHERE vs HAVING

```sql
SELECT COUNT(*), AVG(cgpa), MAX(cgpa), MIN(cgpa), SUM(credits) FROM Student;

SELECT dept, AVG(cgpa)
FROM   Student
GROUP  BY dept
HAVING AVG(cgpa) > 8.0;
```

WHERE	HAVING
Filters **rows** before grouping	Filters **groups** after GROUP BY
Cannot use aggregate functions	Meant for aggregate conditions (AVG(cgpa) > 8)

NULL values

NULL = unknown / missing. Arithmetic with NULL gives NULL. Comparing with `=` or `<>` gives **UNKNOWN** (neither TRUE nor FALSE), so SQL provides `IS NULL` and `IS NOT NULL`.

> ⚠ **Careful**
>
> `WHERE x = NULL` never matches anything. Write `WHERE x IS NULL`.

Nested subqueries

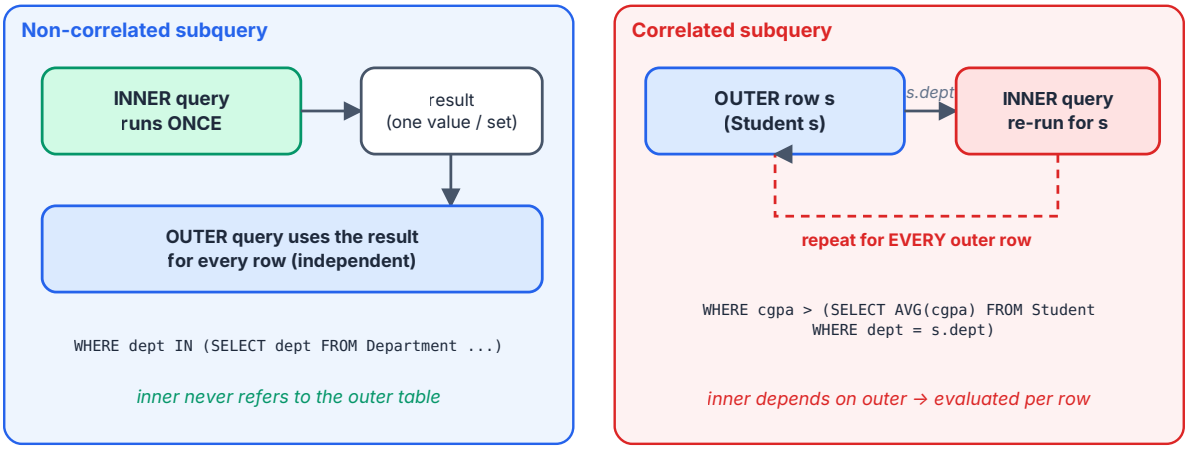


Fig. 19 — Plain vs correlated subquery — how many times the inner query runs

```sql
-- IN
SELECT name FROM Student
WHERE dept IN (SELECT dept FROM Department WHERE hod = 'Dr. Sharma');

-- EXISTS
SELECT name FROM Student s
WHERE EXISTS (SELECT 1 FROM Enrollment e WHERE e.roll_no = s.roll_no);

-- Correlated: inner query uses the outer row
SELECT name FROM Student s
WHERE cgpa > (SELECT AVG(cgpa) FROM Student WHERE dept = s.dept);
```

> 📘 **Definition — Correlated subquery**
>
> An inner query that **depends on a column of the outer query**, so it is logically re-evaluated **once per outer row**. An ordinary subquery runs once, independently.

Derived relations

```sql
SELECT d.dept, d.avgcgpa
FROM  (SELECT dept, AVG(cgpa) AS avgcgpa FROM Student GROUP BY dept) AS d
WHERE d.avgcgpa > 8.0;
```

A **derived relation (inline view)** is a subquery in the FROM clause — a temporary named relation usable for the rest of that query.

Views

```sql
CREATE VIEW HighCgpaStudents AS
SELECT name, dept, cgpa FROM Student WHERE cgpa > 8.5;

SELECT * FROM HighCgpaStudents;     -- used like a normal table
```

Definition

A **view** is a virtual table defined by a stored query; it stores no data (unless materialised) and is recomputed from base tables each time.

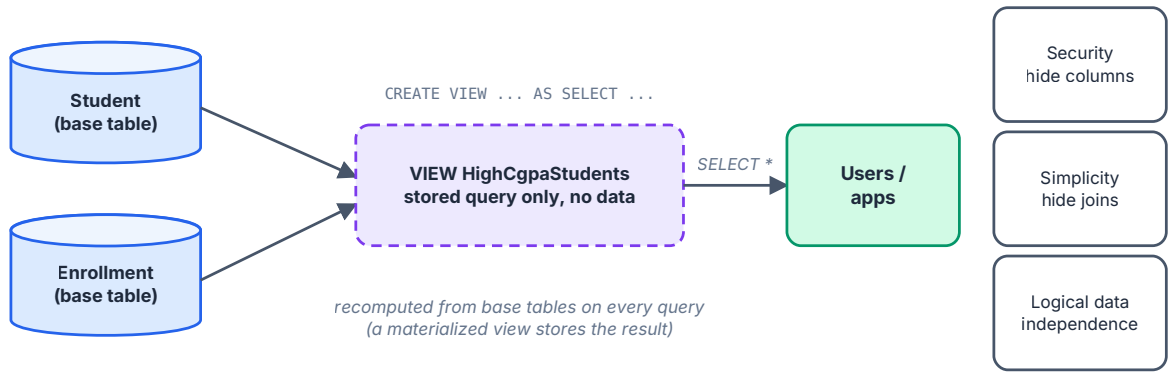


Fig. 20 — A view is a window over base tables — three reasons to use it

1. **Security** — expose only some columns / rows.
2. **Simplicity** — wrap a complex join; users just `SELECT * FROM view`.
3. **Logical data independence** — if tables change, only the view definition is updated.

Modification (DML)

```sql
INSERT INTO Student (roll_no, name, cgpa) VALUES (101, 'Rohit', 9.1);
UPDATE Student SET cgpa = 9.3 WHERE roll_no = 101;
DELETE FROM Student WHERE cgpa < 5.0;
```

Join relations

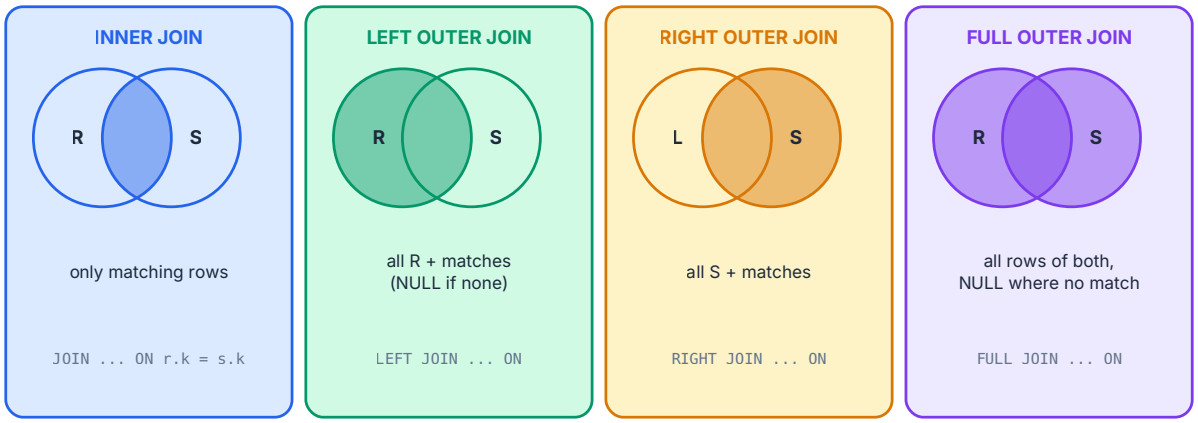


Fig. 21 — INNER, LEFT, RIGHT and FULL OUTER JOIN as Venn diagrams with sample rows

```sql
SELECT s.name, e.course_id
FROM Student s INNER JOIN      Enrollment e ON s.roll_no = e.roll_no;  -- matching rows only
-- LEFT  OUTER JOIN : all of Student + matches (NULL if none)
-- RIGHT OUTER JOIN : all of Enrollment + matches
-- FULL  OUTER JOIN : everything from both, NULLs where unmatched
```

🏭 Real-life use case — Where LEFT JOIN is used

"List **all** students and their courses, including students who enrolled in nothing" → LEFT OUTER JOIN. An INNER JOIN would silently hide those students.

DDL in SQL

```sql
CREATE TABLE Student (
  roll_no INT PRIMARY KEY,
  name    VARCHAR(50) NOT NULL,
  cgpa    DECIMAL(3,2) CHECK (cgpa >= 0 AND cgpa <= 10),
  dept_id INT,
  FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
ALTER TABLE Student ADD COLUMN email VARCHAR(100);
ALTER TABLE Student DROP COLUMN email;
TRUNCATE TABLE Student;   -- all rows, keeps structure
DROP TABLE Student;       -- structure + data
```

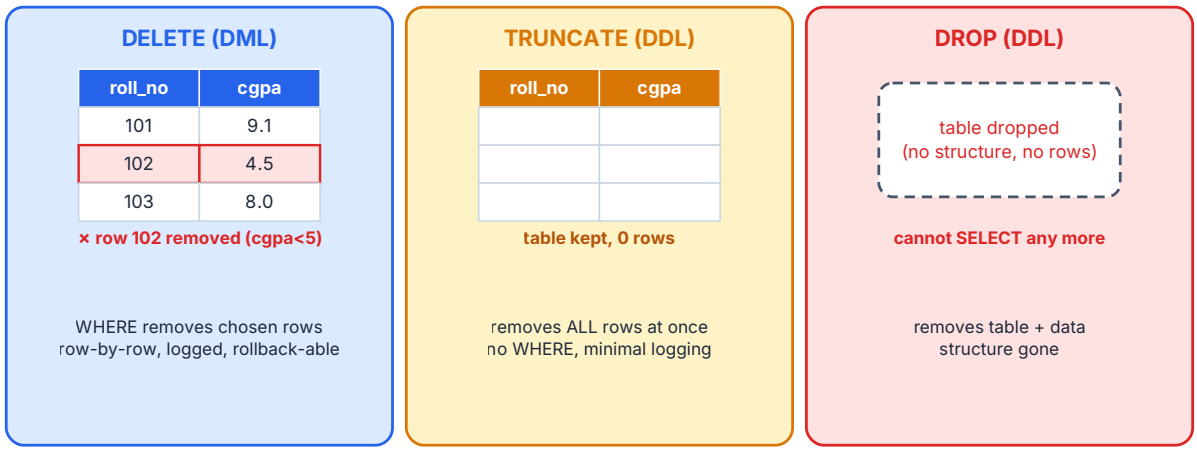


Fig. 22 — DELETE vs TRUNCATE vs DROP — what survives each command

DELETE	TRUNCATE	DROP
DML — rows one by one; WHERE allowed	DDL — all rows at once; no WHERE	DDL — whole table (structure + data)
Slower (row logging); can be rolled back	Faster (minimal logging); often cannot be rolled back	Definition removed permanently
Structure stays	Structure stays	Table no longer exists

🧠 Yaad rakho

DELETE = **kuch rows**, TRUNCATE = **saari rows**, DROP = **table hi gayab**.

📖 GFG: SQL Joins · WHERE vs HAVING · Correlated Subquery · DROP vs TRUNCATE vs DELETE

❓ Likely exam questions — Unit II

- Define degree, cardinality, tuple, relation schema vs instance.
- Explain fundamental operations of relational algebra with examples.
- Union-compatibility. Intersection from set difference.
- Theta vs equijoin vs natural join; what is division?
- TRC vs DRC with examples; algebra vs calculus.
- What is QBE? Write a QBE query.
- Logical order of SQL execution. WHERE vs HAVING.
- NULL handling; UNION vs UNION ALL.
- Correlated subquery; derived relation; views and their uses.
- INNER / LEFT / RIGHT / FULL joins with Venn diagram.
- DROP vs TRUNCATE vs DELETE.

UNIT III · 8 HOURS

Functional Dependencies, Normalization & Integrity

Functional dependency | Closure X⁺ | Armstrong's axioms | Canonical cover | Anomalies | Lossless join | 1NF–5NF | BCNF | Integrity | Triggers

3.1 Functional Dependencies (FD)

Definition

A **functional dependency X → Y** holds on relation R if for any two tuples t1, t2: whenever **t1.X = t2.X**, then **t1.Y = t2.Y**. The value of X uniquely determines the value of Y.

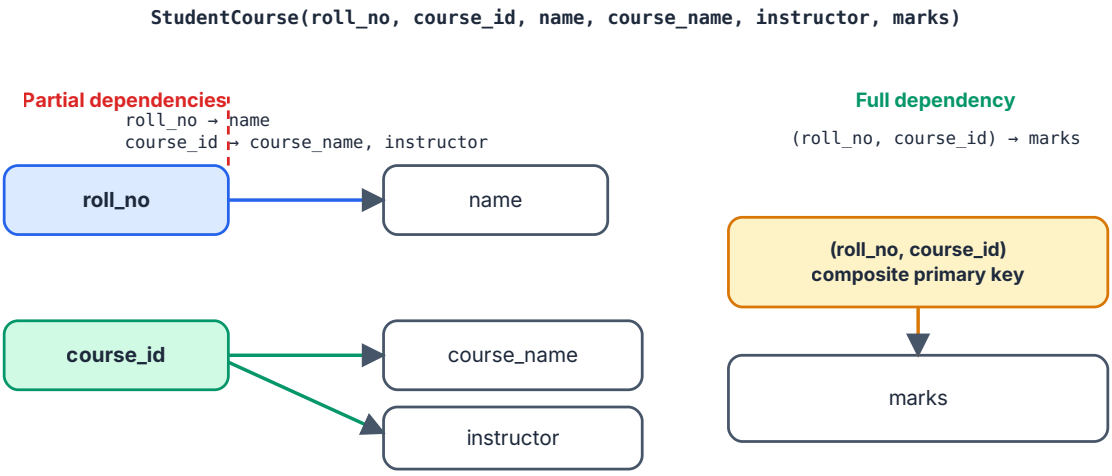


Fig. 23 — FD diagram for StudentCourse — arrows show who determines whom

Example: in `Student(roll_no, name, dept)`, `roll_no → name` holds because a roll number identifies exactly one name.

Trivial vs non-trivial

- **Trivial**: X → Y where Y ⊆ X, e.g. `{roll_no, name} → roll_no`. Always true; gives no new information.
- **Non-trivial**: Y ⊄ X. These are the ones that matter in design.

Closure of an attribute set (X⁺)

Definition

X⁺ is the set of all attributes functionally determined by X under a given set F of FDs.

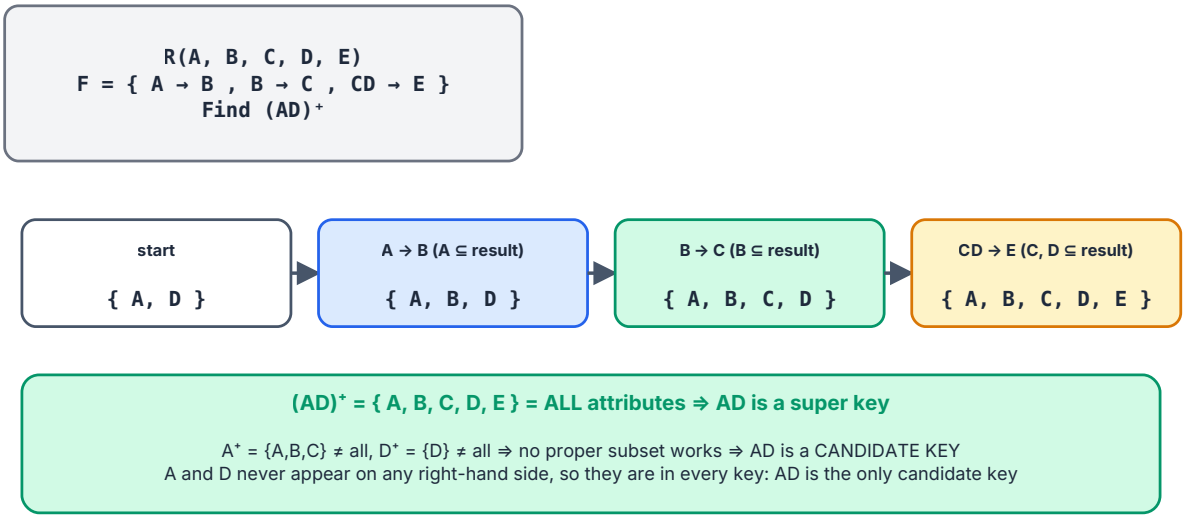


Fig. 24 — Computing X^+ step by step on a worked example

1. Start with **result = X**.
2. For each FD A → B in F: if A ⊆ result, add B to result.
3. Repeat until nothing more can be added.
4. Final result = X^+.

> 💡 **Why it matters — Why closure matters**
>
> (1) FD X → Y is implied by F iff **Y ⊆ X^+**. (2) X is a **candidate key** iff X^+ contains all attributes of R and no proper subset of X does.

> ⭐ **Exam tip**
>
> "Find candidate keys" = compute closures. Attributes that never appear on any RHS **must** be in every key — start from them.

Closure of FDs (F^+) & Armstrong's axioms

> 📘 **Definition**
>
> **F^+** is the set of all FDs logically implied by F, computed with **Armstrong's axioms**.

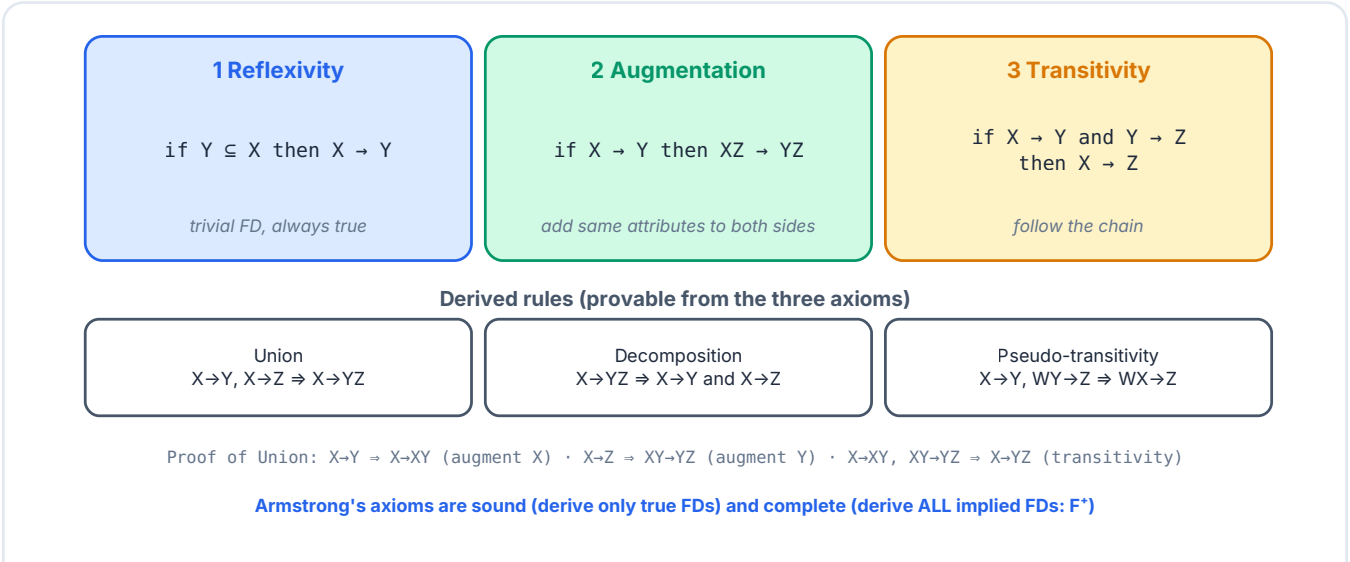


Fig. 25 — Armstrong's three axioms and the three derived rules

Rule	Statement
Reflexivity	If Y ⊆ X then X → Y
Augmentation	If X → Y then XZ → YZ
Transitivity	If X → Y and Y → Z then X → Z
Union *(derived)*	X → Y and X → Z ⇒ X → YZ
Decomposition *(derived)*	X → YZ ⇒ X → Y and X → Z
Pseudo-transitivity *(derived)*	X → Y and WY → Z ⇒ WX → Z

🧠 **Yaad rakho**

R-A-T: Reflexivity, Augmentation, Transitivity. Baaki teen (Union, Decomposition, Pseudo-transitivity) inhi se bante hain.

Irreducible (canonical / minimal) set of FDs

📘 **Definition**

F is **irreducible** if: (1) every FD has a **single attribute on the RHS**; (2) no FD is **redundant** (removing it changes F⁺); (3) no attribute on any LHS is **extraneous**.

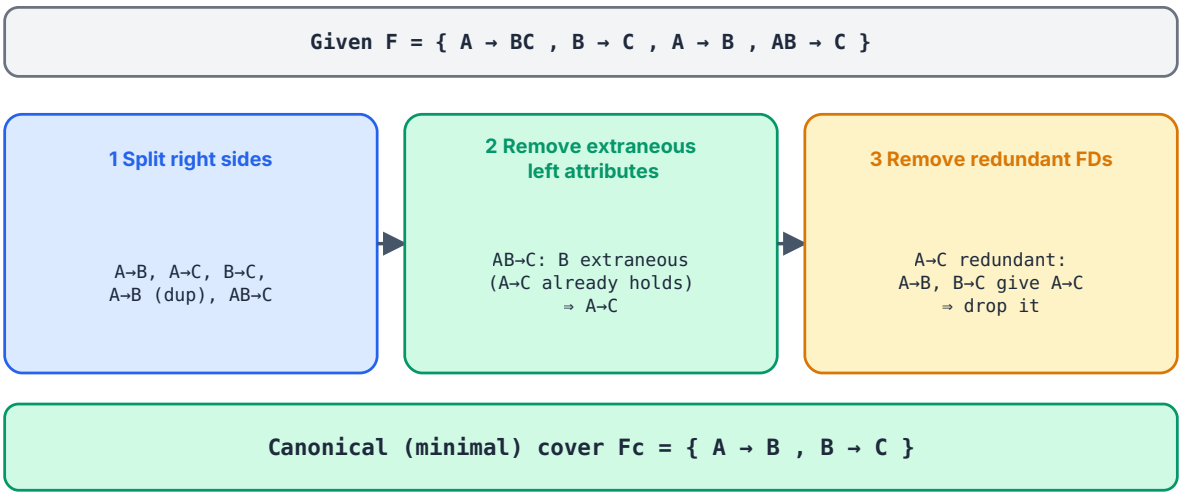


Fig. 26 — Three steps to reach the canonical cover

Why: it strips redundancy so that design decisions (candidate keys, dependency-preservation tests in §3.3) rest only on essential dependencies.

📖 GFG: Functional Dependency · Attribute Closure · Armstrong's Axioms · Canonical Cover

3.2 Introduction to Normalization

📘 Definition

Normalization is the systematic process of organising attributes and tables to **minimise redundancy** and avoid **insertion, deletion and update anomalies**, by decomposing relations according to their functional dependencies.

The three anomalies

roll_no	name	course_id	course_name	instructor
101	Rohit	C1	ADBMS	Dr. Rao
102	Anjali	C1	ADBMS	Dr. Rao
103	Kunal	C2	Java	Dr. Iyer

Insertion anomaly

Cannot add new course C3 until some student enrols in it (roll_no would have to be NULL)

Deletion anomaly

Delete the only student of C2 (Kunal) → the fact 'C2 = Java, Dr. Iyer' is lost too

Update anomaly

Change Dr. Rao's name: must edit EVERY row of C1; miss one → inconsistent data

Fix: decompose into Student(roll_no, name) · Course(course_id, course_name, instructor) · Enrollment(roll_no, course_id) so every fact is stored exactly once

Fig. 27 — One unnormalised table, three kinds of trouble

Take `StudentCourse(roll_no, name, course_id, course_name, instructor)`:

Anomaly	What goes wrong
Insertion	Cannot add a new course until at least one student enrols (course data lives only inside a student's row)
Deletion	Deleting the only student of a course also erases the course's name and instructor
Update	Changing an instructor means editing every row of that course; missing one leaves inconsistent data

Fix: decompose into **Student**, **Course** and **Enrollment** so each fact is stored in exactly one place.

🏭 **Real-life use case — Real-life symptom**

An e-commerce "orders" table that repeats the customer's address on every order: change the address once and half the old rows still show the old one — an update anomaly.

Lossless (non-loss) decomposition

📘 **Definition**

Decomposing R into R1 and R2 is **lossless** if the natural join R1 ⋈ R2 gives back exactly R — no spurious tuples, no information lost.

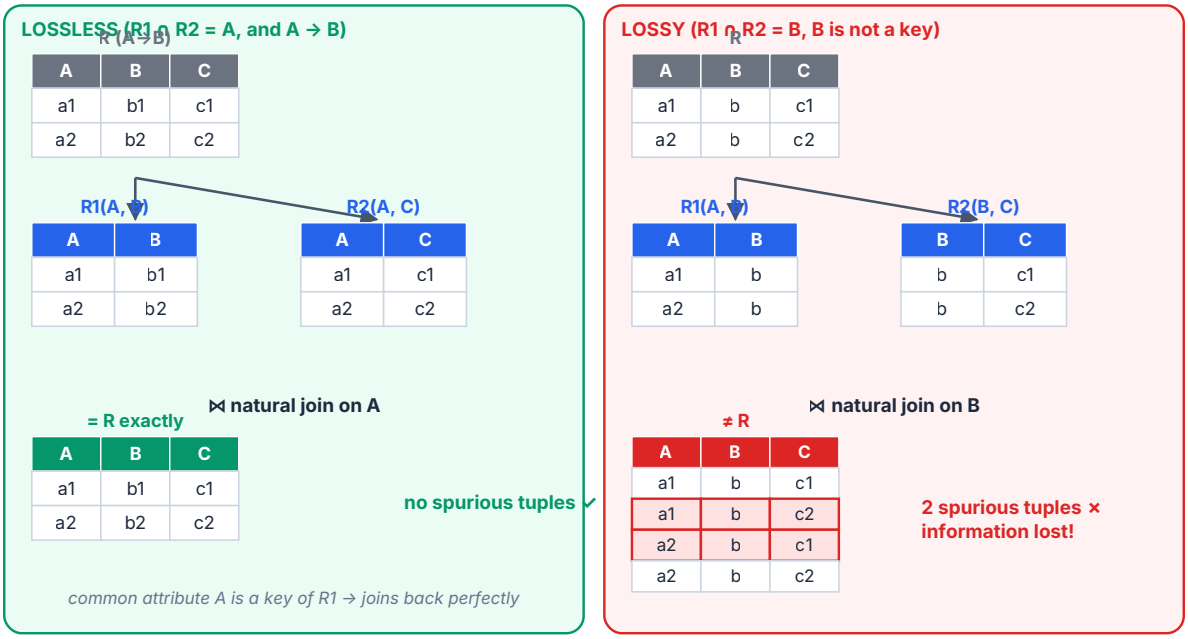


Fig. 28 — A lossless split vs a lossy split of the same table

⭐ **Exam tip — Exact condition — memorise**

R → R1, R2 is lossless iff **(R1 ∩ R2) → R1** OR **(R1 ∩ R2) → R2**. The common attributes must be a key of at least one part.

📖 GFG: Normalization · Anomalies · Lossless Join

3.3 Normal Forms

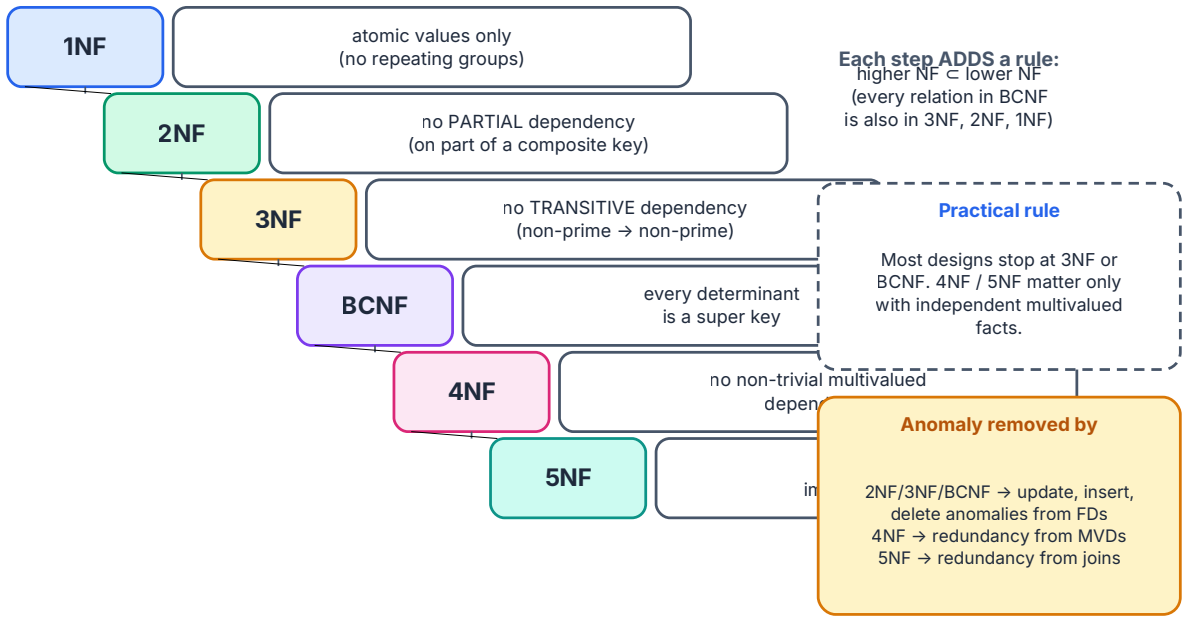


Fig. 29 — The normal-form ladder — each step removes one kind of problem

1NF — First Normal Form

📘 **Definition**

A relation is in **1NF** if every attribute holds only **atomic** values — no multi-valued or composite attributes, no repeating groups.

Violation: `phone_numbers = "9876543210, 9123456789"`. Fix: a separate table `(roll_no, phone_number)`, one row per phone.

2NF — Second Normal Form

📘 **Definition**

In **1NF** and every non-prime attribute is **fully** functionally dependent on the whole primary key — **no partial dependency**.

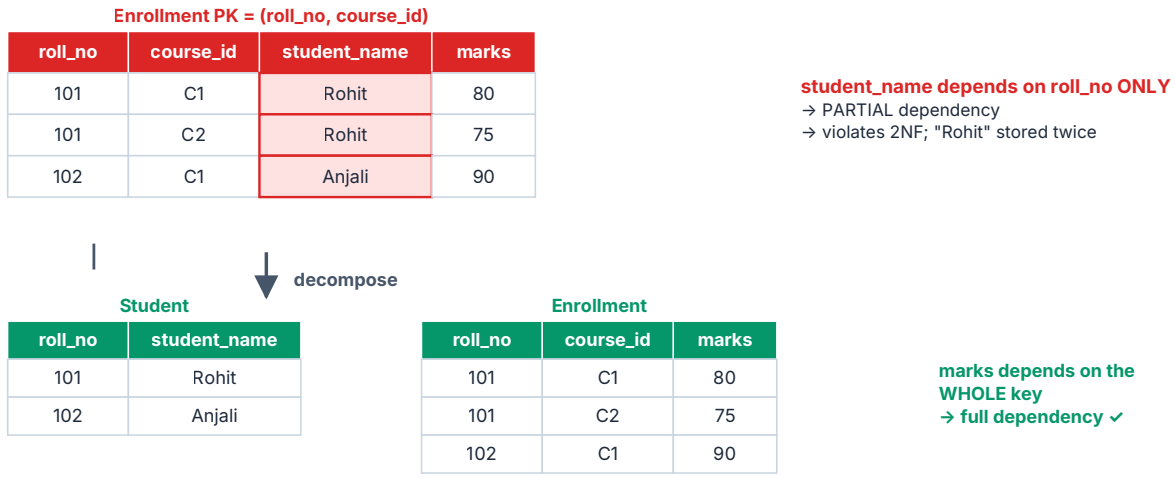


Fig. 30 — Partial dependency and its fix

Matters only for **composite** keys. With a single-attribute key, 1NF ⇒ 2NF automatically.

```
TEXT
Enrollment(roll_no, course_id, student_name, marks)   PK = (roll_no, course_id)
student_name depends only on roll_no  -> PARTIAL dependency -> not 2NF
Fix:  Student(roll_no, student_name)   Enrollment(roll_no, course_id, marks)
```

3NF — Third Normal Form

> 📘 **Definition**
>
> In **2NF** and no non-prime attribute depends on another non-prime attribute — **no transitive dependency**.

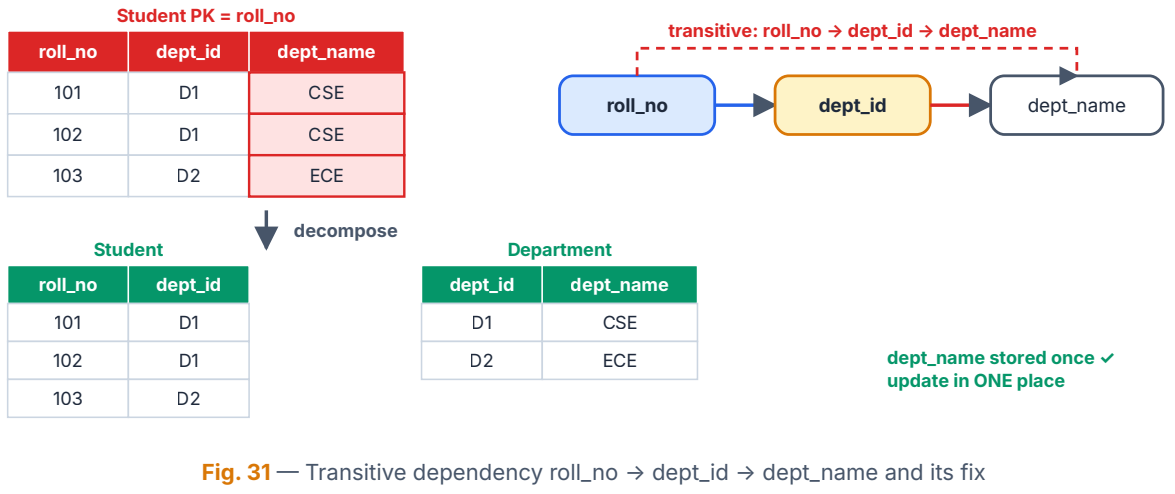


Fig. 31 — Transitive dependency roll_no → dept_id → dept_name and its fix

```
TEXT
Student(roll_no, dept_id, dept_name)   PK = roll_no
roll_no -> dept_id -> dept_name  => transitive => not 3NF
Fix:  Student(roll_no, dept_id)   Department(dept_id, dept_name)
```

> ⭐ **Exam tip — Formal 3NF — quote for full marks**
>
> For every non-trivial FD X → A in R, at least one holds: **(a) X is a super key**, OR **(b) A is a prime attribute**. This handles edge cases that "no transitive dependency" misses.

Dependency preservation

> 📘 **Definition**
>
> A decomposition is **dependency preserving** if every FD of the original set F can be enforced using only the FDs of the decomposed relations, without joining them back.

Why: otherwise every update would need an expensive join just to check an FD.

BCNF — Boyce-Codd Normal Form

> 📘 **Definition**
>
> R is in **BCNF** if for every non-trivial FD X → A in R, **X is a super key** of R.

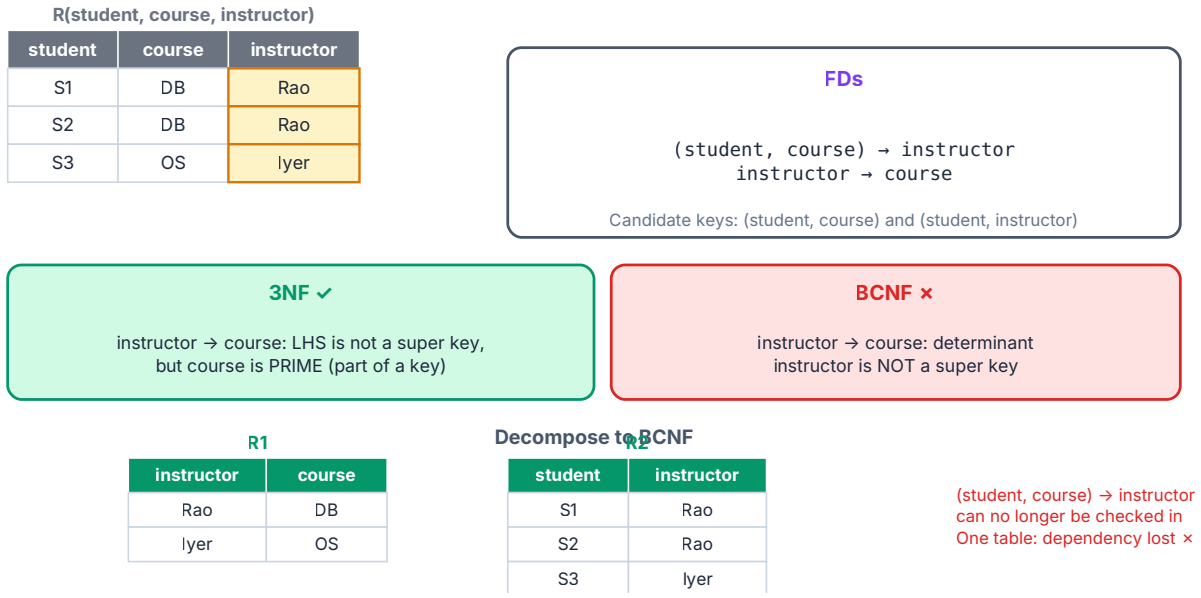


Fig. 32 — A relation in 3NF but not BCNF, and the BCNF split

3NF	BCNF
Allows X → A if X is a super key **or** A is prime	X must **always** be a super key — no prime-attribute exception
Can always be reached **with** dependency preservation	May sacrifice dependency preservation
Weaker	Stricter: BCNF ⇒ 3NF, not vice versa

> ⭐ **Exam tip**
>
> "Example of 3NF but not BCNF" → relation with **overlapping composite candidate keys**, where a non-superkey determines part of another key.

4NF — Multivalued dependency

Definition

A **multivalued dependency X →→ Y** holds if, for a given X, the set of Y values is **independent** of the remaining attributes. **4NF**: in BCNF and no non-trivial MVD (except with a super-key LHS).

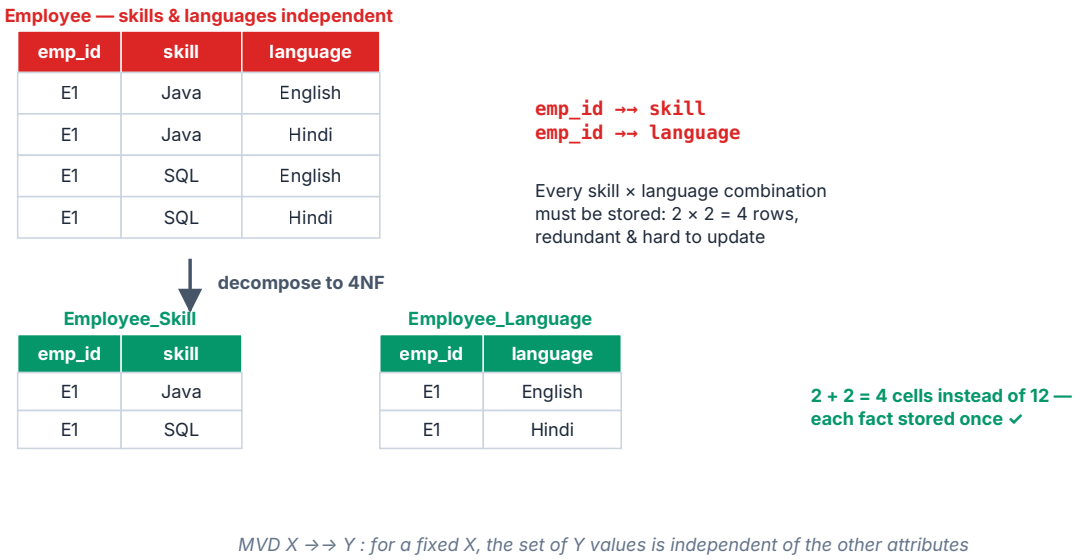


Fig. 33 — Employee skills and languages are independent — a 4NF violation, then the fix

`Employee(emp_id, skill, language)` forces every skill × language combination (`emp_id →→ skill`, `emp_id →→ language`). Fix: `Employee_Skill(emp_id, skill)` and `Employee_Language(emp_id, language)`.

5NF — Join dependency

Definition

A **join dependency *(R1,...,Rn)** holds if R can be losslessly rebuilt by joining R1...Rn. **5NF (Project-Join NF)**: in 4NF and every join dependency is implied by the candidate keys.

5NF is for relations that can only be decomposed losslessly into **three or more** parts (not two) — questions are usually conceptual.

Summary table — memorise whole

NF	Removes	Condition
1NF	Repeating groups / non-atomic values	All values atomic
2NF	Partial dependency	No non-prime attribute depends on part of a composite key
3NF	Transitive dependency	No non-prime attribute depends on another non-prime attribute
BCNF	Anomalies from non-superkey determinants	Every determinant is a super key
4NF	Multivalued-dependency redundancy	No non-trivial MVD except super-key LHS
5NF	Join-dependency redundancy	Every JD implied by candidate keys

🧠 **Yaad rakho**

"**The key, the whole key, and nothing but the key — so help me Codd.**" 2NF = the *whole* key, 3NF = *nothing but* the key.

📖 GFG: Normal Forms 1NF–5NF · BCNF vs 3NF · Multivalued Dependency & 4NF

3.4 Database Integrity

📘 **Definition**

Database integrity is the correctness, consistency and validity of stored data, enforced through **integrity constraints**.

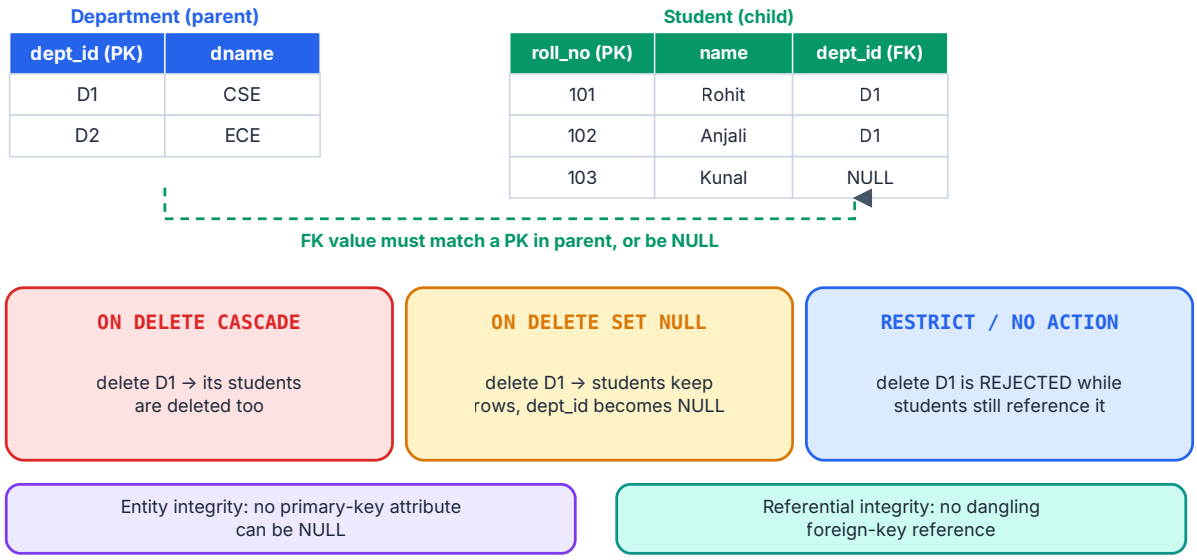


Fig. 34 — Entity, referential, domain and attribute rules in one picture

The two fundamental rules

1. **Entity integrity** — no attribute of a primary key may be NULL.
2. **Referential integrity** — a foreign-key value must match an existing primary key in the parent relation, or be NULL; it can never point to a non-existent row.

Domain rules

Restrict the legal values of an attribute via its data type plus extra restrictions.

```sql
cgpa DECIMAL(3,2) CHECK (cgpa BETWEEN 0 AND 10)
```

Attribute rules

```sql
name  VARCHAR(50)  NOT NULL,
email VARCHAR(100) UNIQUE,
age   INT          CHECK (age >= 18)
```

NOT NULL = value required · **UNIQUE** = no duplicates · **CHECK** = custom condition · **DEFAULT** = used when no value is given.

Assertions

📘 Definition

An **assertion** is a general constraint expressed as a predicate the database must **always** satisfy; unlike CHECK (one table / column) it can span **multiple tables**.

```sql
CREATE ASSERTION total_credit_check
CHECK ( NOT EXISTS (
  SELECT roll_no FROM Enrollment
  GROUP BY roll_no
  HAVING SUM(credits) > 30 ) );
```

⭐ Exam tip — Shows deeper understanding

Assertions are important in theory but **rarely supported by commercial DBMSs** — triggers are used instead. Mention this in the exam.

Triggers

📘 Definition

A **trigger** is a stored procedure executed automatically by the DBMS in response to an **INSERT, UPDATE or DELETE** event on a table.

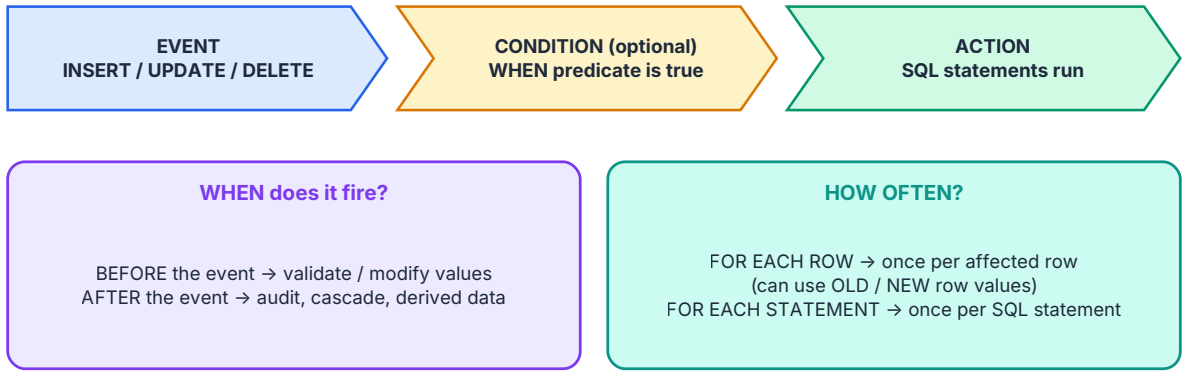


Fig. 35 — Event → Condition → Action, and BEFORE/AFTER, ROW/STATEMENT options

```sql
CREATE TRIGGER before_student_delete
BEFORE DELETE ON Student
FOR EACH ROW
BEGIN
  INSERT INTO Student_Audit(roll_no, deleted_on) VALUES (OLD.roll_no, NOW());
END;
```

Part	Meaning
Event	The INSERT / UPDATE / DELETE that fires the trigger
Condition	Optional predicate; action runs only if true
Action	The SQL statements executed (logging, business rules, maintaining derived data)

Triggers fire **BEFORE** or **AFTER** the event, and **FOR EACH ROW** (once per affected row) or **FOR EACH STATEMENT** (once per statement) — frequently tested.

🏭 **Real-life use case — Audit trail**

Banks use triggers to log every change to an account balance into an audit table — nobody can change a balance silently.

Integrity in SQL — ON DELETE / ON UPDATE

```sql
CREATE TABLE Enrollment (
  roll_no   INT,
  course_id INT,
  PRIMARY KEY (roll_no, course_id),
  FOREIGN KEY (roll_no) REFERENCES Student(roll_no)
      ON DELETE CASCADE ON UPDATE CASCADE,
  FOREIGN KEY (course_id) REFERENCES Course(course_id)
);
```

Action	Effect on child rows
CASCADE	Delete / update matching child rows automatically
SET NULL	Set the child's foreign key to NULL
RESTRICT / NO ACTION	Reject the parent delete / update while children exist

📖 GFG: Integrity Constraints · SQL Triggers · ON DELETE CASCADE

❓ **Likely exam questions — Unit III**

- Define FD; trivial vs non-trivial.
- Compute attribute closure X^+ / find candidate keys.
- State and prove using Armstrong's axioms.
- Find the canonical (minimal) cover of F.
- Three anomalies with example.
- Lossless decomposition and its condition.
- Normalise a table up to 3NF / BCNF.
- 3NF vs BCNF; example of 3NF but not BCNF.
- 4NF (MVD) and 5NF (join dependency).
- Entity vs referential integrity; assertions; triggers.
- ON DELETE CASCADE / SET NULL / RESTRICT.

UNIT IV · 8 HOURS

Transactions, Concurrency, Recovery & Query Processing

ACID | Transaction states | Shadow paging & WAL | Concurrency problems | Serializability | 2PL & protocols | Deadlock | Failures & storage | Log-based recovery | Checkpoints | Query optimisation

4.1 Transactions & ACID Properties

Definition

A **transaction** is a sequence of read / write operations treated as a single, indivisible logical unit of work — it must complete entirely or have **no effect at all**.

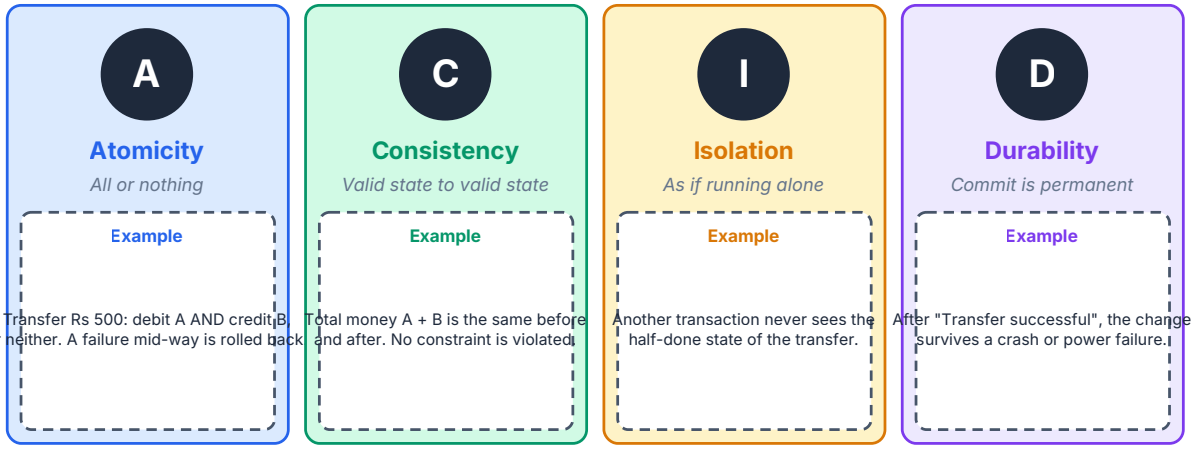


Fig. 36 — ACID — four guarantees, each with the bank-transfer example

Property	Meaning
Atomicity	All-or-nothing. If a transaction fails midway, all its effects so far are undone (rolled back).
Consistency	Run alone on a consistent database, it leaves the database consistent — no integrity constraint is violated.
Isolation	Though many run concurrently, each appears to run alone; the partial state of one is invisible to others.
Durability	Once committed, changes persist permanently, even after a crash or power failure.

Exam tip

For a 5-mark "explain ACID": definition of each **plus one tiny example each** (Atomicity → debit and credit both happen, or neither).

🧠 Yaad rakho

Kaun kisko dekhta hai: **Atomicity** → recovery manager, **Consistency** → programmer + DBMS constraints, **Isolation** → concurrency-control manager, **Durability** → recovery manager.

Transaction states

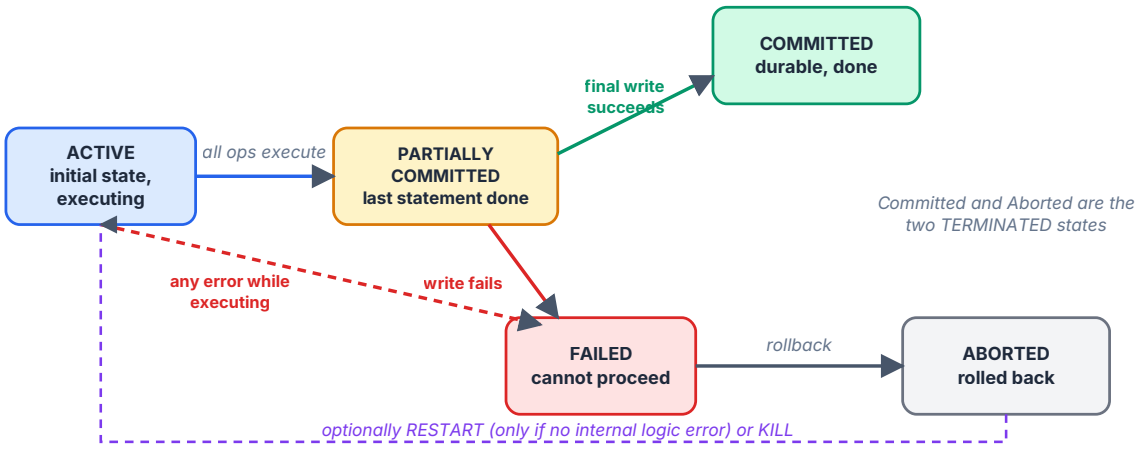


Fig. 37 — State diagram: Active → Partially committed → Committed / Failed → Aborted

A transaction starts **Active**. After its last statement it is **Partially committed** (computed, not yet guaranteed on stable storage). If the final write succeeds → **Committed**. If anything fails at any point → **Failed** → rolled back to **Aborted**, restoring the state before the transaction. From Aborted it may be **restarted** (if no internal logic error) or **killed**.

Implementing atomicity & durability

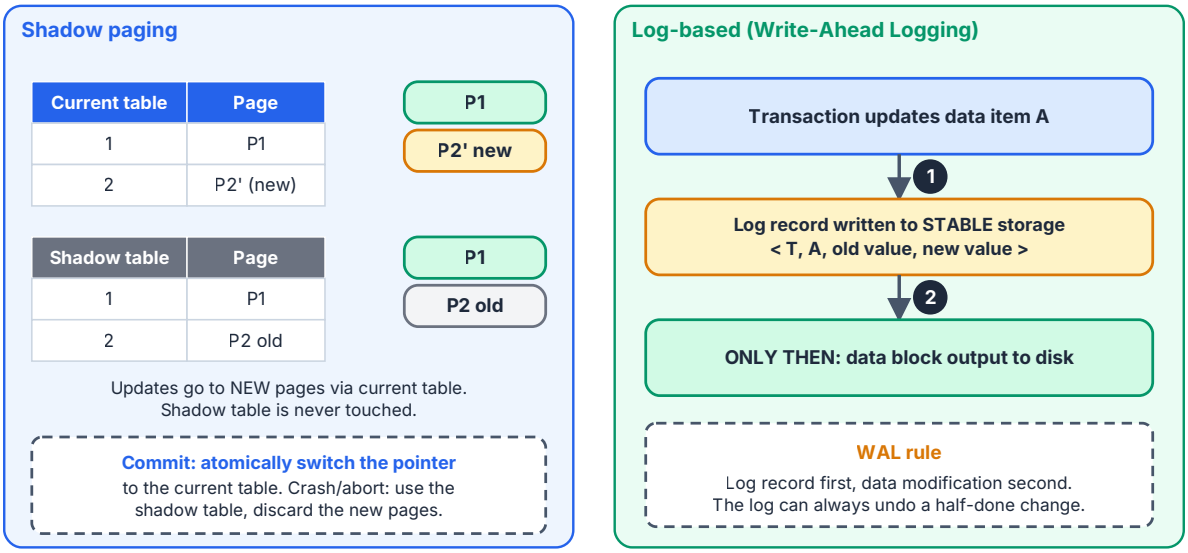


Fig. 38 — Shadow paging vs write-ahead logging

1. **Shadow copy / shadow paging** — updates go to a shadow copy; at commit a pointer is switched atomically. On failure the untouched original is used.

2 **Log-based recovery (WAL)** — the most widely used. A log record is written to stable storage **before** the database change; after a crash the log is used to redo committed or undo uncommitted work.

⭐ **Exam tip — The WAL rule — state it exactly**

A log record for an update must be written to **stable storage BEFORE** the corresponding database modification is written to disk.

📖 GFG: ACID Properties · Transaction States · Write-Ahead Logging

4.2 Concurrent Execution

📘 **Definition**

Concurrent execution means several transactions are processed together (interleaved) rather than one after another, to improve **throughput** and CPU / disk utilisation.

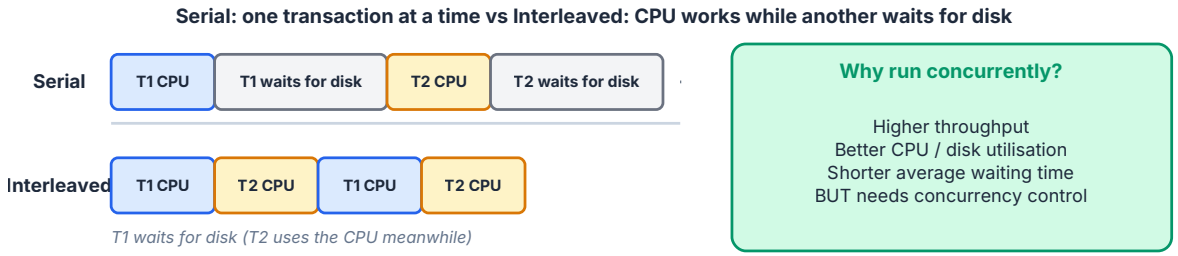


Fig. 39 — Serial vs interleaved — the CPU works while another transaction waits for disk

Problems from uncontrolled concurrency

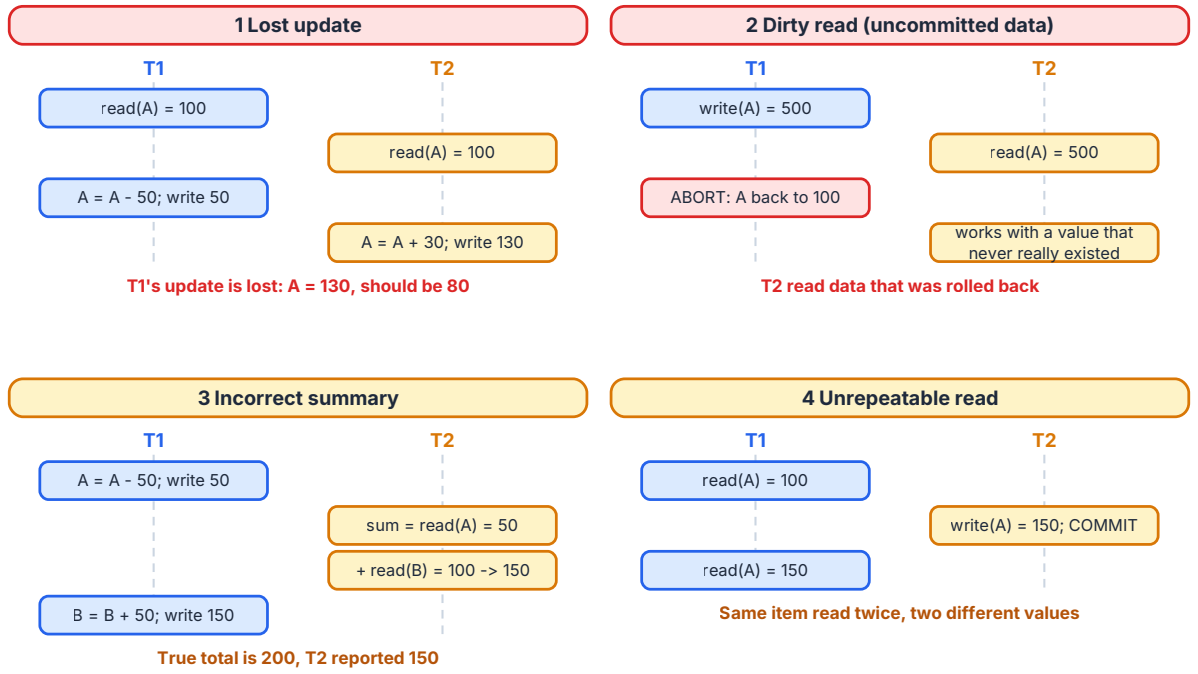


Fig. 40 — The four classic problems as two-transaction timelines

Problem	What happens
Lost update	Two transactions read the same item, both write; the second write overwrites the first and its update is lost
Dirty read (temporary update)	T2 reads a value written by T1 *before T1 commits*; if T1 aborts, T2 read a value that never existed
Incorrect summary	One transaction moves money between accounts while another computes SUM over them — inconsistent total
Unrepeatable read	T1 reads an item twice and gets different values because T2 committed an update in between

🏭 **Real-life use case — Ticket booking**

Two users book the last seat at the same time: both read "1 seat left", both write "0 left" and both get a ticket — a **lost update**. Locking (§4.4) is what prevents it.

📖 GFG: Concurrency Problems

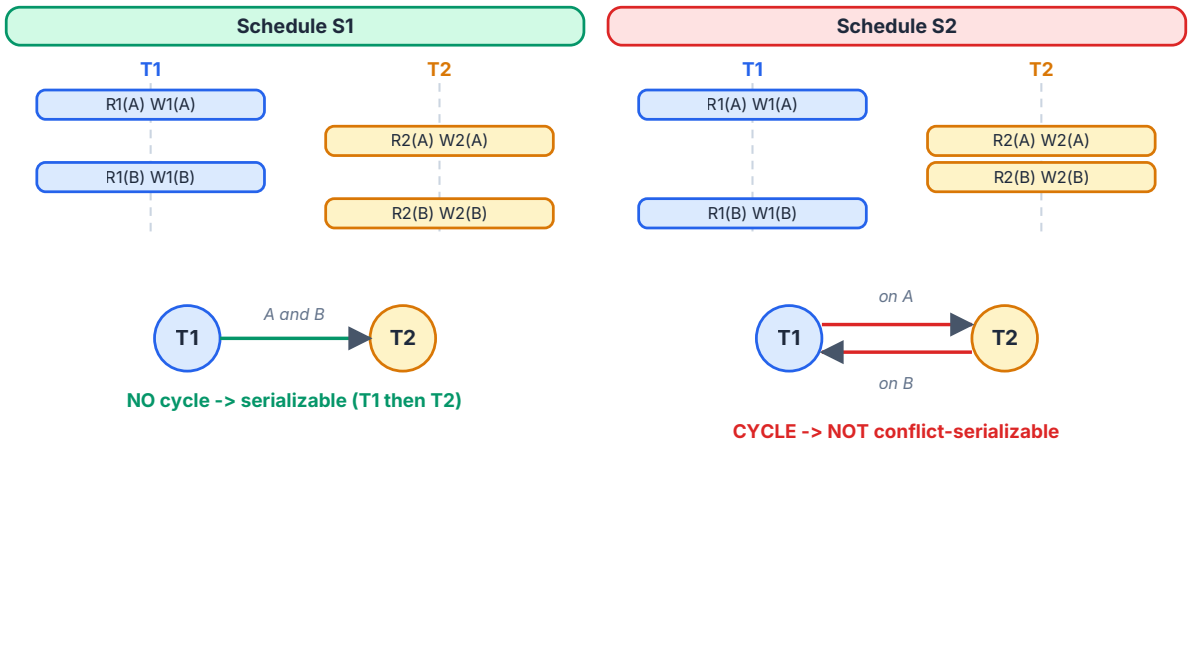
4.3 Serializability

📘 **Definition**

A concurrent schedule is **serializable** if it produces the same final result as **some serial schedule** of the same transactions — the primary correctness criterion.

Type	Idea
Conflict serializability	Schedule can be turned into a serial one by swapping **non-conflicting** operations. Two operations conflict if they are from different transactions, on the **same item**, and **at least one is a write**.
View serializability	View-equivalent to a serial schedule: same initial reads, same read-from relationships, same final writer per item. More general.

Every conflict-serializable schedule is view-serializable, but **not vice versa**.



Edge Ti -> Tj when a conflicting op of Ti comes BEFORE one of Tj on the same item (different txns, same item, at least one write)

Fig. 41 — Precedence graph: acyclic ⇒ conflict-serializable, cycle ⇒ not

1. Make a node for each transaction.
2. For each pair of conflicting operations, draw edge **Ti → Tj** if Ti's operation comes first.
3. Look for a cycle. **No cycle ⇒ conflict-serializable** (a topological order gives the serial schedule). **Cycle ⇒ not**.

> ⭐ **Exam tip — Most common numerical**
>
> Practise drawing the precedence graph for a given schedule and spotting cycles. Conflicts: **R-W, W-R, W-W** on the same item (R-R never conflicts).

📖 GFG: Conflict Serializability & Precedence Graph

4.4 Concurrency Control

> 📘 **Definition**
>
> **Concurrency control** is the set of protocols a DBMS uses to make concurrent execution produce a **serializable** (correct) schedule while maximising concurrency.

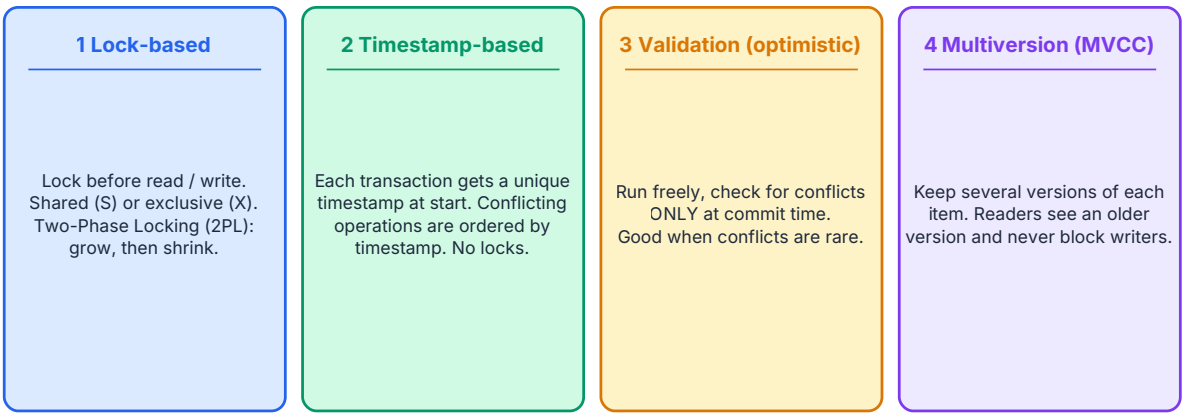


Fig. 42 — The four families of concurrency-control protocols

Protocol	Idea
Lock-based	Acquire shared / exclusive locks before read / write. **Two-phase locking (2PL)**: growing phase (only acquire), then shrinking phase (only release).
Timestamp-based	Each transaction gets a unique start timestamp; conflicting operations are ordered by it — no locks.
Validation (optimistic)	Execute freely; check for conflicts only at commit. Good when conflicts are rare.
Multiversion (MVCC)	Keep several versions of each item so readers get an older version and do not block writers.

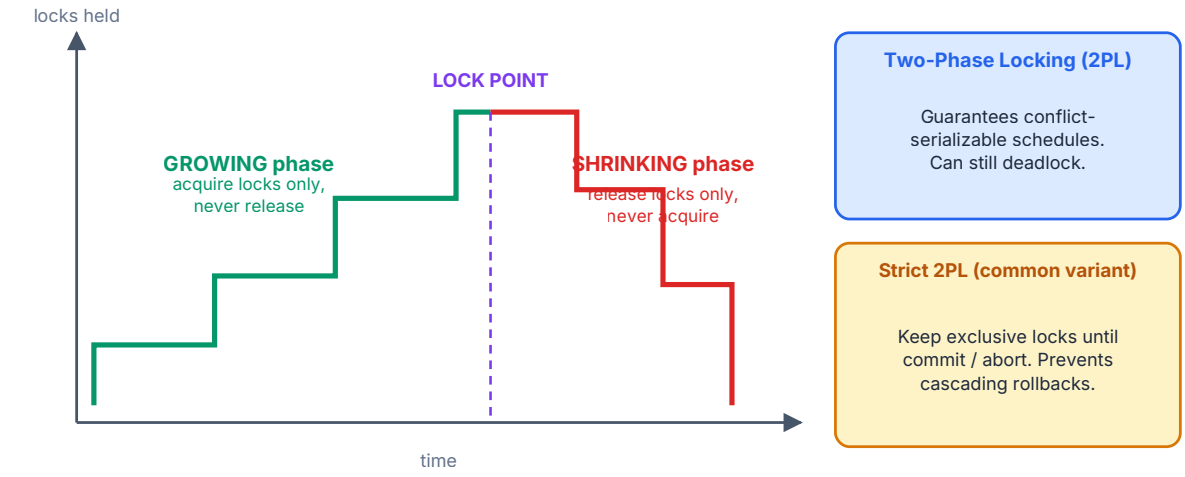


Fig. 43 — Two-phase locking: lock point separates growing and shrinking phases

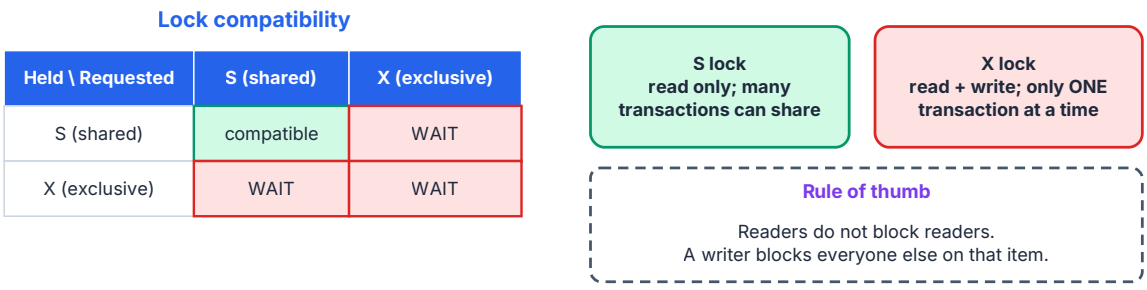


Fig. 44 — Lock compatibility: only S with S is allowed

🏭 Real-life use case — MVCC in the real world

PostgreSQL and Oracle use MVCC: a long report query keeps reading its own snapshot while other users keep updating — readers never block writers.

🧠 Yaad rakho

2PL: **pehle sirf lock lo (grow), phir sirf chhodo (shrink)** — ek baar chhod diya to dobara lock nahi.

📖 GFG: Concurrency Control · Two-Phase Locking

4.5 Deadlock

📘 Definition

A **deadlock** is a situation where two or more transactions each wait for a lock held by another in the group, forming a **cycle of waiting**; none can proceed.

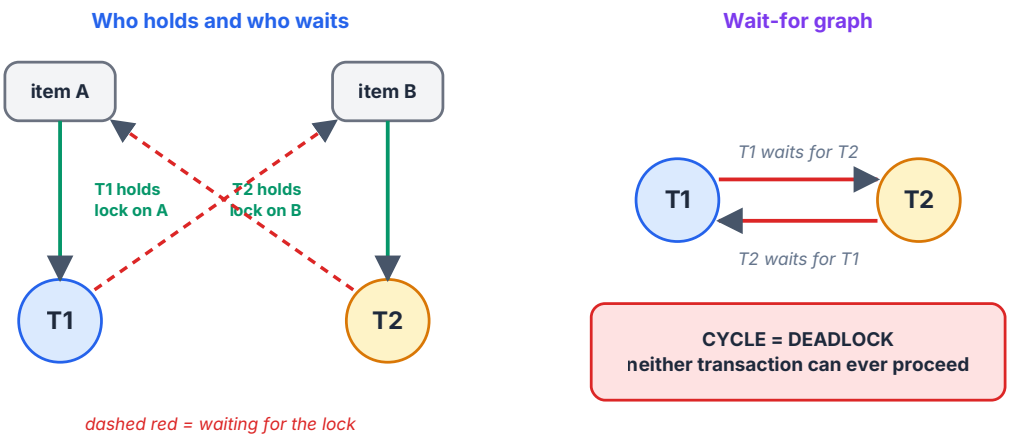


Fig. 45 — T1 holds A and wants B; T2 holds B and wants A — circular wait and its wait-for graph

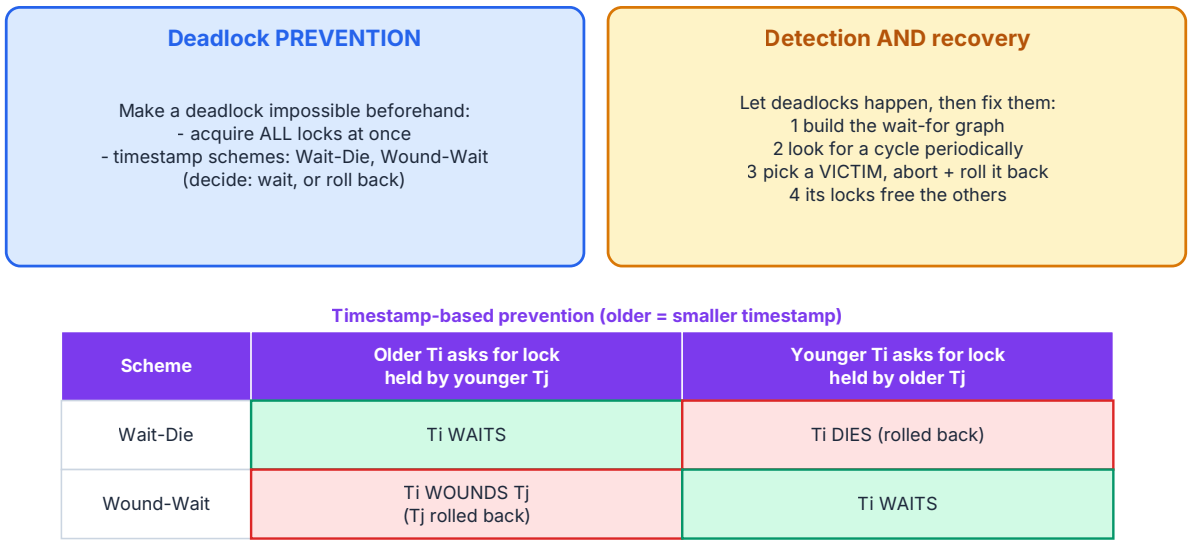


Fig. 46 — Prevention vs detection & recovery, with Wait-Die / Wound-Wait

Approach	How
Prevention	Make deadlock impossible: acquire all locks at once, or use timestamp schemes **Wait-Die** and **Wound-Wait** to decide wait or rollback.
Detection & recovery	Allow deadlocks; periodically build the **wait-for graph** (Ti → Tj: Ti waits for Tj). A **cycle** = deadlock. Pick a **victim**, abort and roll it back to release its locks.

⭐ **Exam tip — Wait-Die vs Wound-Wait**

Older = smaller timestamp. **Wait-Die**: older waits, younger dies. **Wound-Wait**: older wounds (aborts) the younger, younger waits. Both are non-preemptive / preemptive respectively.

📖 GFG: Deadlock in DBMS · Wait-Die & Wound-Wait

4.6 Failure Classification

📘 **Definition**

A **failure** is any event that stops a transaction or the whole system from running correctly.

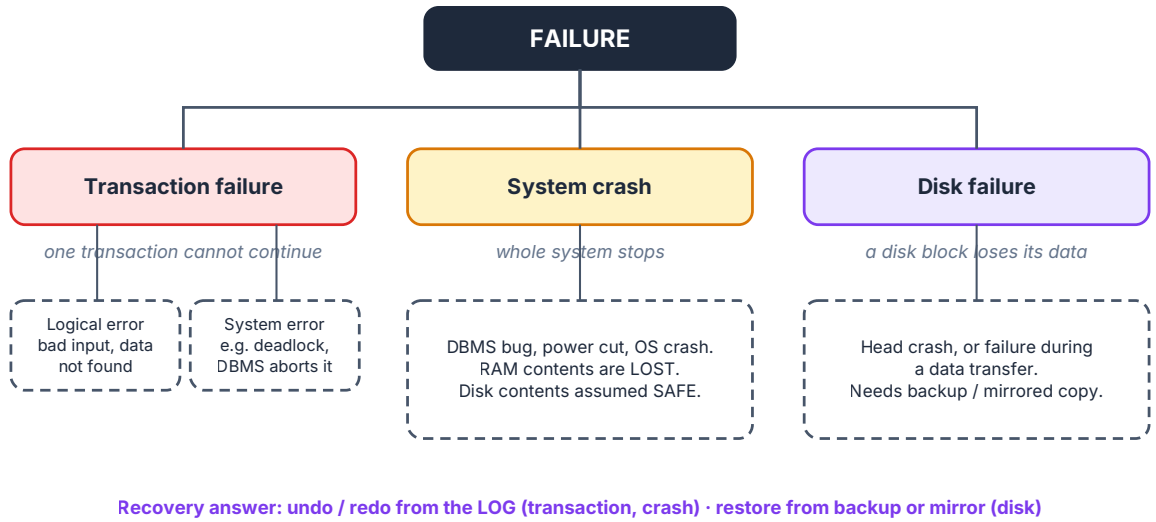


Fig. 47 — Three failure classes and what is lost in each

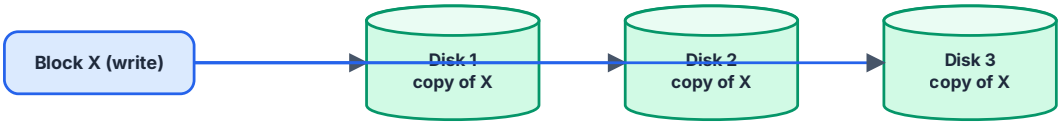
Failure	Detail
Transaction failure	*Logical error* — bad input, data not found, resource limit. *System error* — e.g. deadlock; DBMS aborts the transaction.
System crash	Hardware / software fault (DBMS bug, power cut) stops the system; non-volatile storage is assumed intact.
Disk failure	A disk block loses data — head crash or failure during a data transfer.

4.7 Storage Types & Stable Storage

Storage classified by volatility

Storage type	Examples	Survives system crash?	Survives disk failure?
Volatile	RAM, cache	No	No
Nonvolatile	Disk, SSD, tape	Yes	No (may be lost)
Stable (idealised)	Mirrored disks (RAID), remote copies	Yes	Yes

Implementing stable storage: replicate every block on separate nonvolatile media



One disk fails -> the other copies still hold the data

Fig. 48 — Volatile, non-volatile and stable storage, and how stable storage is implemented

Type	Survives crash?	Example
Volatile	No	RAM, cache
Non-volatile	Yes (but a disk failure can still lose it)	Disk, tape
Stable	Assumed to survive **everything**	Replicated copies — RAID mirroring, remote copies

Stable storage is an **idealisation**: in practice the same information is replicated on several physically separate non-volatile media so one failure cannot lose it.

4.8 Data Access — Buffer Management

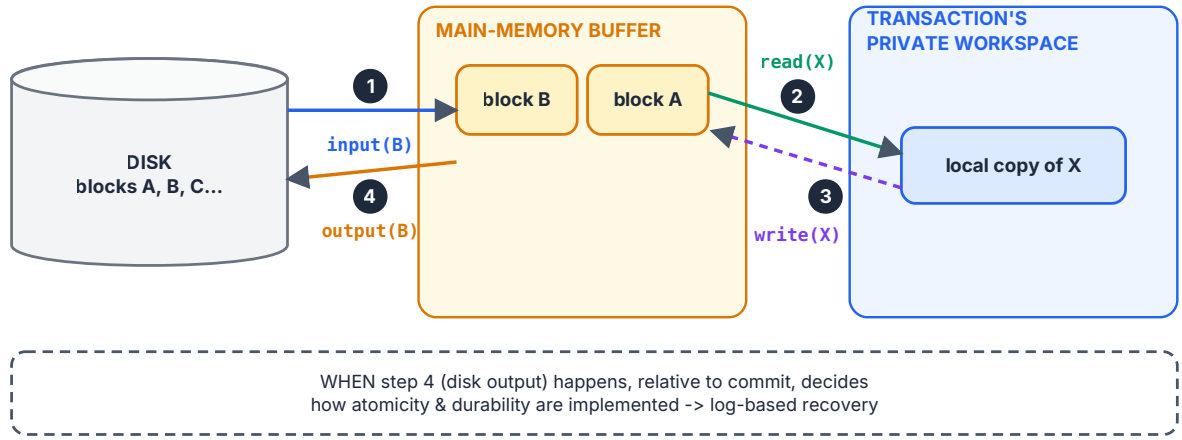


Fig. 49 — Blocks move disk → buffer (input) → workspace (read/write) → disk (output)

Data lives on disk in fixed-size **blocks**. To work on it a block is **input** into a memory buffer; after modification it is **output** back to disk. Each transaction keeps a **local copy** of items in its private workspace. *When* the output happens relative to commit decides how logging must guarantee atomicity and durability (§4.9).

4.9 Log-Based Recovery

📘 **Definition**

Log-based recovery keeps a sequential, append-only **log on stable storage** with a record for every update, used to restore the database to a consistent state after a crash.

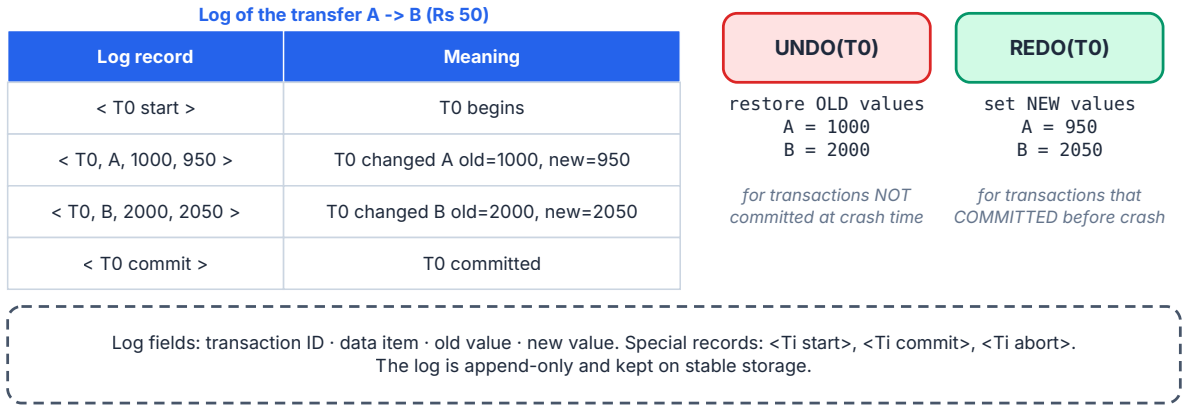


Fig. 50 — Log of a transfer A → B (Rs 50) and what UNDO / REDO do with it

Log record fields: transaction ID · data-item name · **old value** · **new value**; plus special records `<Ti start>`, `<Ti commit>`, `<Ti abort>`.

Procedure	Meaning
UNDO(Ti)	Restore every item updated by Ti to its **old value** — for transactions **not committed** at the crash
REDO(Ti)	Set every item updated by Ti to its **new value** — for transactions that **committed** but whose changes may not have reached disk

Deferred vs immediate modification

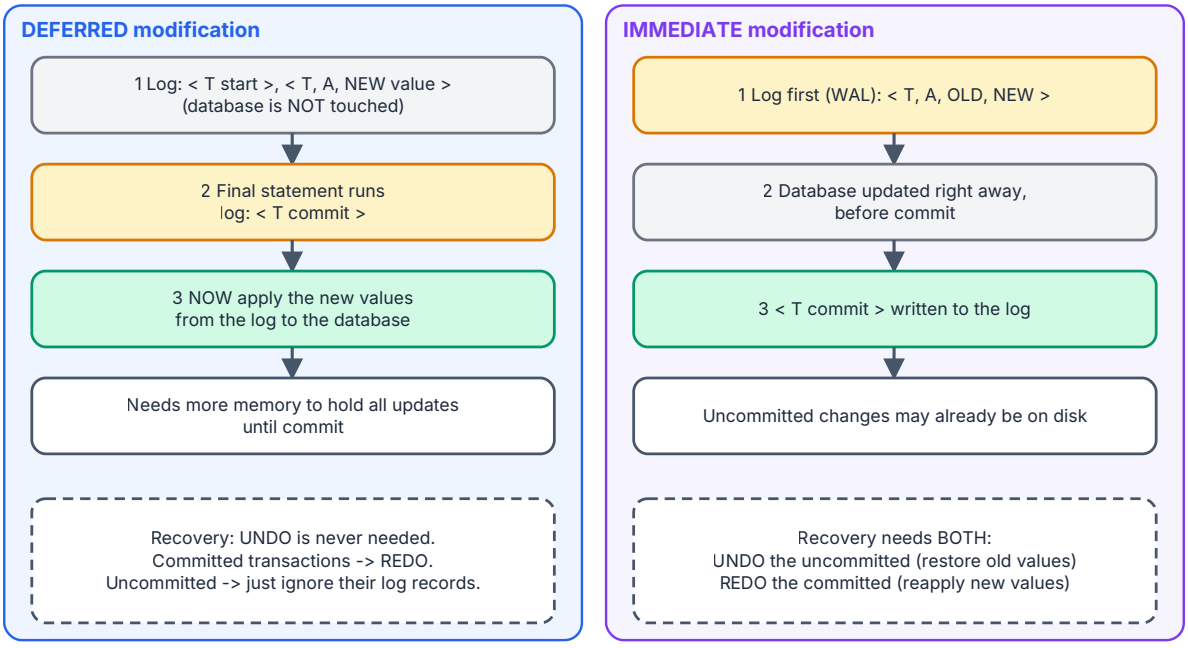


Fig. 51 — Two schemes side by side — when the database is actually touched

Deferred modification	Immediate modification
Updates only in the log until commit; DB changed **after** commit	DB updated **during** execution; log (old + new values) written first (WAL)
Recovery needs **REDO only**	Recovery needs **both UNDO and REDO**
Simpler recovery, but more memory to hold updates	Less memory, more complex recovery

Checkpoints

📘 Definition

A **checkpoint** periodically writes all buffered log records and modified data to stable storage and then writes a `<checkpoint>` record, so recovery need not scan the whole log.

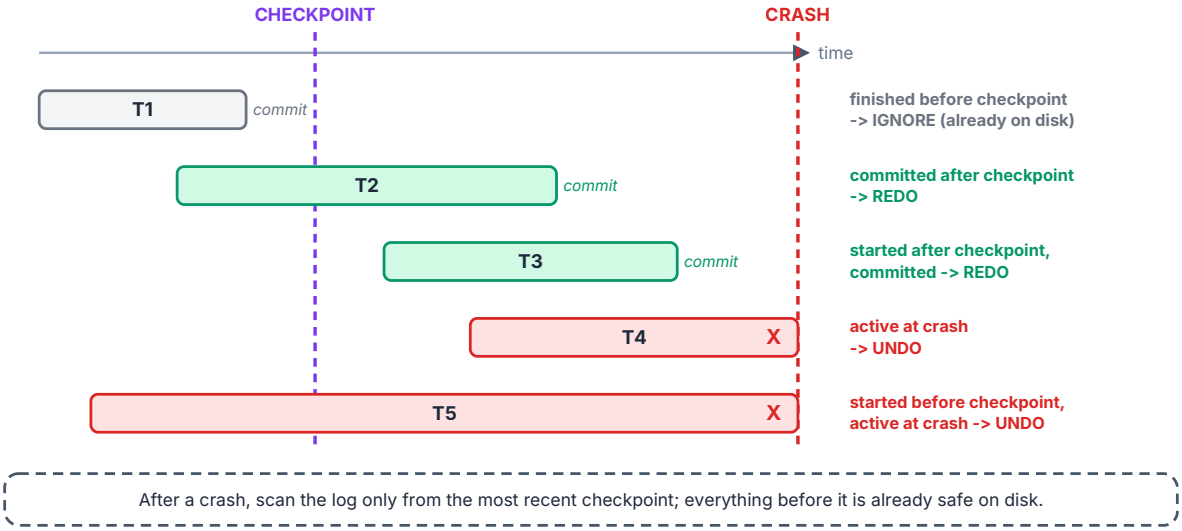


Fig. 52 — Which transactions need UNDO, REDO, or nothing after a crash

After a crash only transactions **active at or after the last checkpoint** are considered; everything before it is already safe on disk.

🧠 Yaad rakho

Commit ho gaya → **REDO**. Commit nahi hua → **UNDO**. Checkpoint se pehle commit → **kuch nahi**.

📖 GFG: Log-Based Recovery · Checkpoints

4.10 Query Processing & Optimization

📘 Definition

Query processing is the series of steps a DBMS takes to turn a high-level SQL query into an efficient low-level sequence of operations that produces the result.

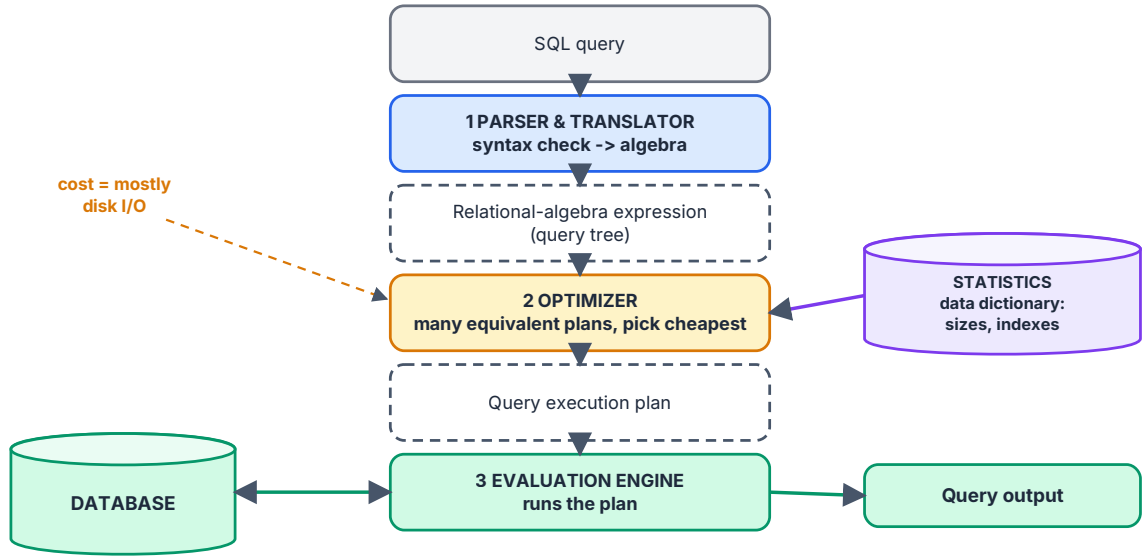


Fig. 53 — Parser & translator → optimizer → evaluation engine

1. **Parsing & translation** — check syntax, translate to an internal form (relational-algebra expression tree).
2. **Optimization** — generate many equivalent plans, estimate the **cost** of each (table sizes, indexes, selectivity) and pick the cheapest.
3. **Evaluation** — the engine executes the chosen plan using low-level operators (join algorithms, index scan vs full scan).

One SQL query maps to many equivalent algebra expressions (push σ below a join or not), and each operation has several algorithms (nested-loop, hash, sort-merge join). The optimizer searches this space for the lowest cost — mostly **disk I/O**.

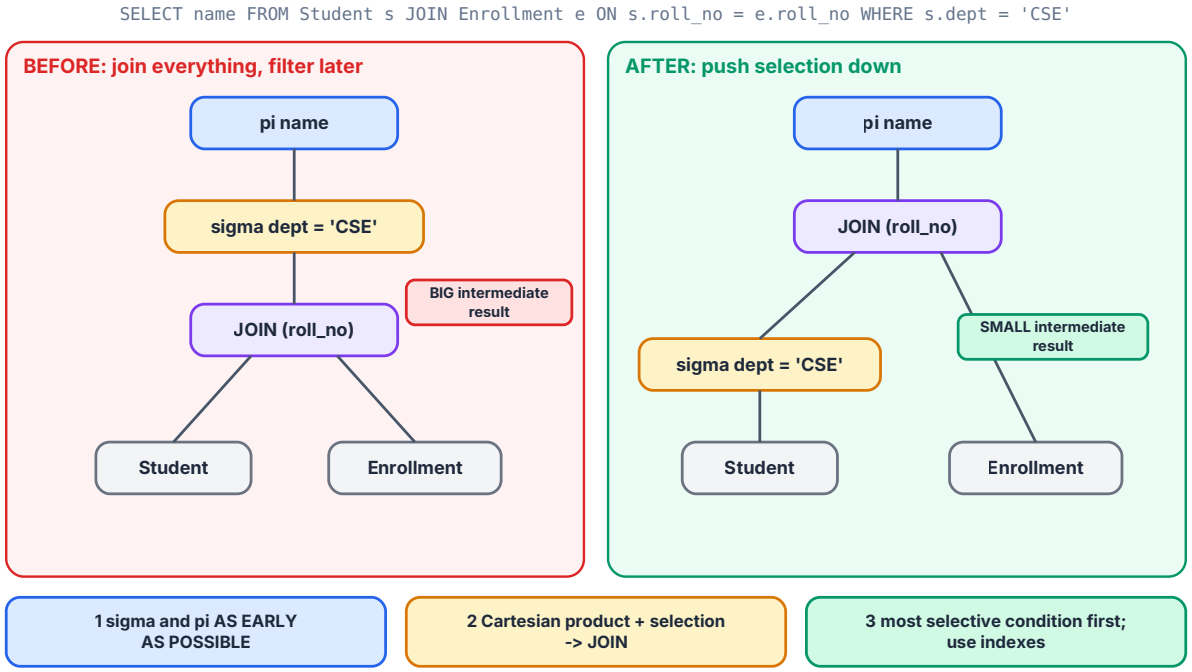


Fig. 54 — Pushing selection below the join shrinks intermediate results

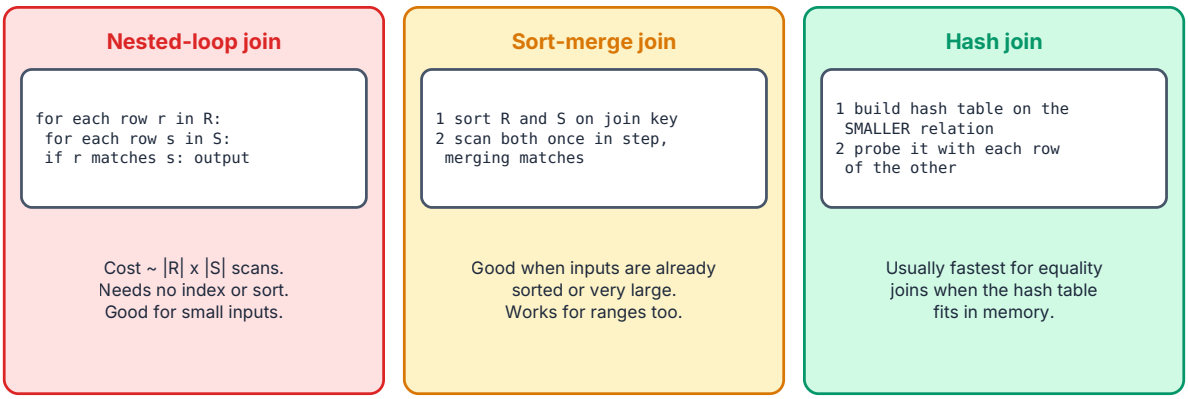


Fig. 55 — Three join algorithms and when each wins

Key heuristics

- Do **selection (σ) and projection (π) as early as possible** — shrinks intermediate relations before joins.
- **Combine Cartesian product + selection into a join** — far cheaper than a full product.
- Apply the **most selective** conditions first and use available **indexes**.

🏭 Real-life use case — EXPLAIN in practice

Run `EXPLAIN SELECT ...` in PostgreSQL / MySQL to see the plan the optimizer chose — if you see a full table scan on a big table, an index is probably missing.

📖 GFG: Query Processing · Query Optimization

❓ Likely exam questions — Unit IV

- Explain ACID with examples; draw the transaction state diagram.
- Shadow paging vs log-based atomicity; the WAL rule.
- Lost update, dirty read, incorrect summary, unrepeatable read.
- Conflict vs view serializability; test a schedule with a precedence graph.
- Categories of concurrency control; two-phase locking.
- Deadlock: prevention vs detection & recovery; wait-for graph; Wait-Die / Wound-Wait.
- Failure classification; volatile / non-volatile / stable storage.
- Deferred vs immediate modification (UNDO / REDO).
- Checkpoints — why needed?
- Steps of query processing; heuristics of query optimisation.

UNIT V

Graph, Spatial, Temporal Databases & NoSQL

Property graph | Index-free adjacency | Spatial types & queries | R-tree / Quad-tree | Valid vs transaction time | CAP theorem | 4 NoSQL types | SQL vs NoSQL | ACID vs BASE

5.1 Graph Databases

Definition

A **graph database** is a NoSQL database that stores data as a graph of **nodes** (entities), **edges** (relationships) and **properties** (key-value attributes on nodes / edges), built to make traversing relationships fast and natural.

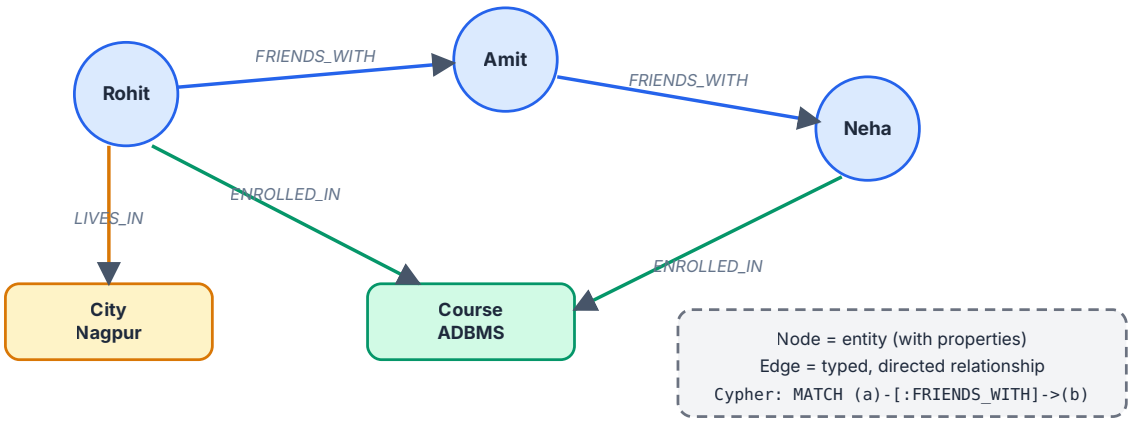


Fig. 56 — Property graph — nodes, typed edges and a Cypher query

- **Node** — an entity (Person, Product) with labels and properties.
- **Edge** — a directed, named connection, e.g. `(Person)-[:FRIENDS_WITH]->(Person)` ; can carry properties (since: 2020).
- **Property graph model** — most common; both nodes and edges hold key-value properties.

Why graph, not relational?

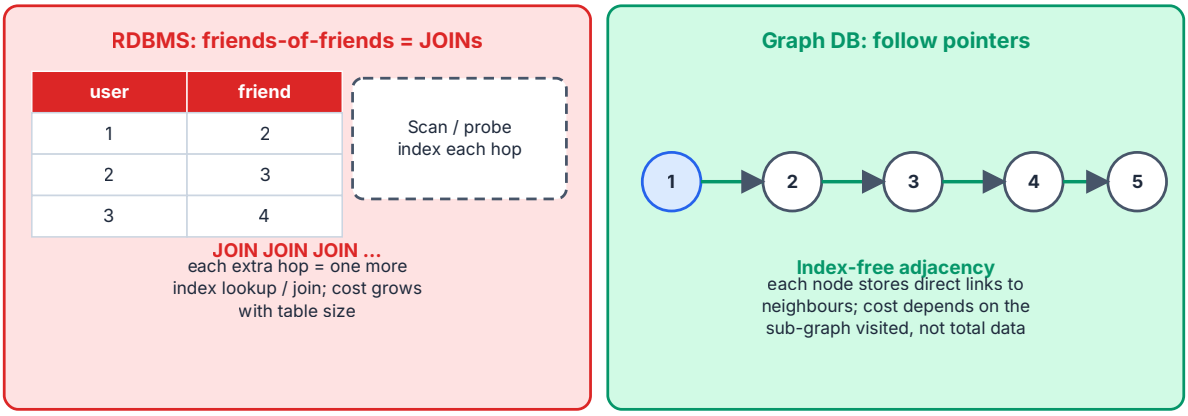


Fig. 57 — Friends-of-friends: many JOINs vs following direct links

In an RDBMS, "friends up to 3 levels deep" needs several expensive JOINs, growing with depth. A graph database stores relationships as **first-class edges** physically linking nodes, so a hop is a direct pointer-follow — **index-free adjacency**.

```cypher
MATCH (p:Person)-[:FRIENDS_WITH]->(friend)
WHERE p.name = 'Rohit'
RETURN friend.name;
```

Popular: **Neo4j** (query language **Cypher**); also Gremlin and SPARQL (RDF graphs).

🏭 Real-life use case

Social networks (friend suggestions) · fraud detection (suspicious transaction rings) · recommendation engines · IT / network management · route and navigation optimisation.

🧠 Yaad rakho

Graph DB = **relationships ko pehli class** ka darja. Join ki jagah **pointer**.

📖 GFG: Graph Databases

5.2 Spatial Databases

📘 Definition

A **spatial database** is optimised to store, query and manipulate **geometric / geographic data** — points, lines, polygons — with specialised indexing and spatial query operations.

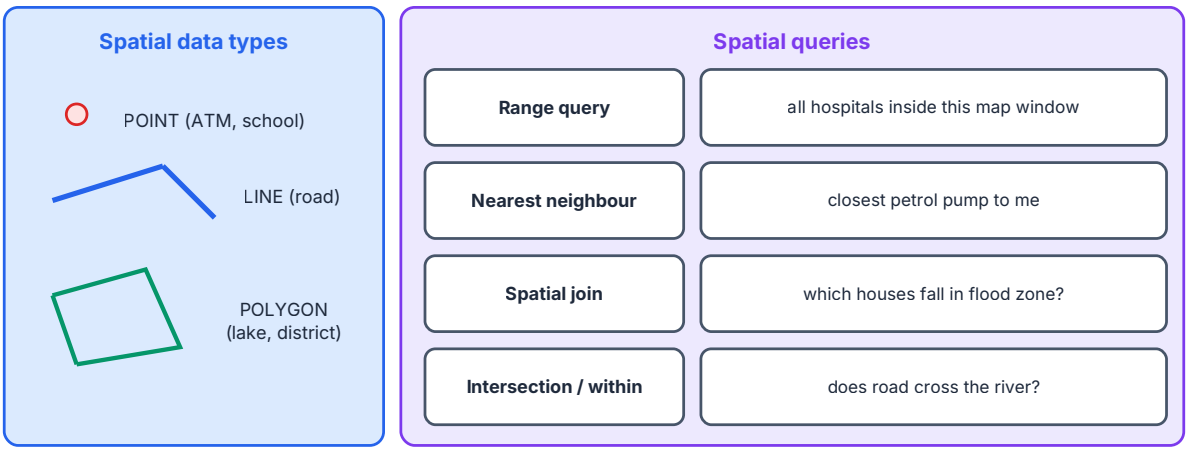


Fig. 58 — Point, line, polygon and the four kinds of spatial queries

Data type	Example
Point	GPS coordinate (latitude, longitude)
Line / LineString	A road or route
Polygon	A country boundary, building footprint

Usually provided via an extension such as **PostGIS** on PostgreSQL.

Spatial query	Example
Distance	How far is this restaurant from me?
Range	All hospitals within 5 km
Nearest neighbour	3 nearest petrol pumps
Topological	contains, intersects, overlaps, touches — which district contains this point?

Spatial indexing

A **B-tree** works for one-dimensional ordered data. Spatial data is **multi-dimensional**, so special indexes are used.

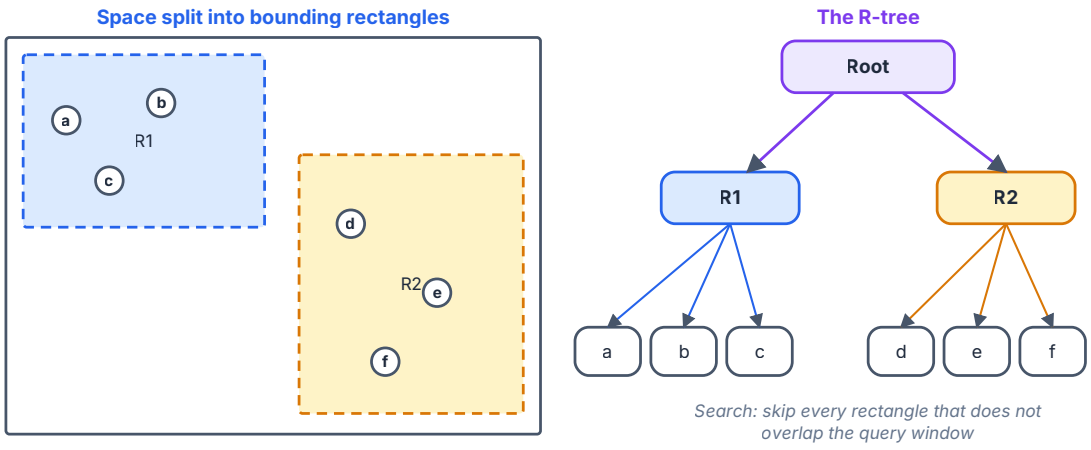


Fig. 59 — R-tree: hierarchical bounding rectangles let a search skip whole regions

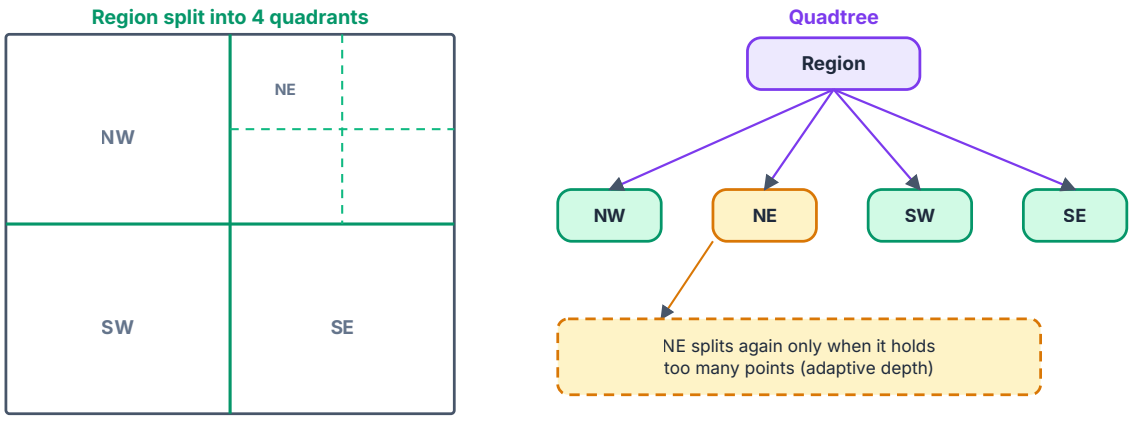


Fig. 60 — Quad-tree: recursive division of 2D space into four quadrants

Index	Idea
R-tree	Groups nearby objects in bounding rectangles arranged hierarchically; discard rectangles that do not overlap the query area
Quad-tree	Recursively splits 2D space into four quadrants; for point / region data

🏭 Real-life use case

GIS · Google-Maps style navigation · urban planning · environmental monitoring · ride-hailing and logistics (find the nearest driver).

📖 GFG: Spatial Databases, R-tree, Quad-tree

5.3 Temporal Databases

📘 Definition

A **temporal database** manages **time-varying data**: it records not just the current state but **when facts were true**, allowing queries about past (and sometimes planned future) states.

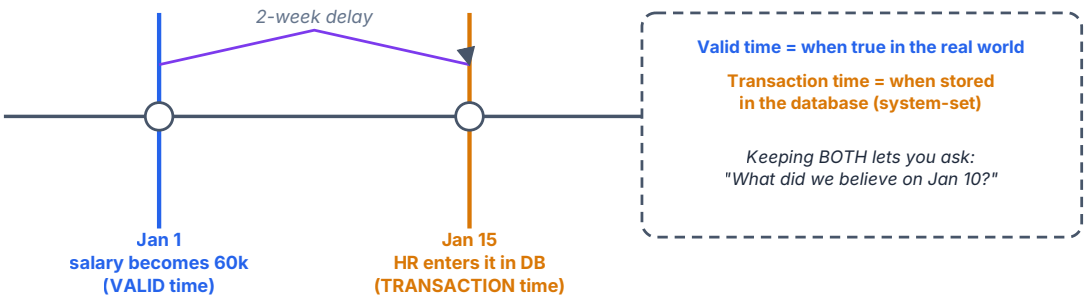


Fig. 61 — Salary rise effective 1 Jan, entered on 15 Jan — valid time vs transaction time

Time	Meaning
Valid time	Period a fact is true in the **real world**, independent of when it was recorded
Transaction time	Period a fact was stored in the **database** as the current official value

Example: a raise effective **1 January** is entered on **15 January**. Valid time starts 1 Jan; transaction time starts 15 Jan. A temporal DB keeps both timelines.

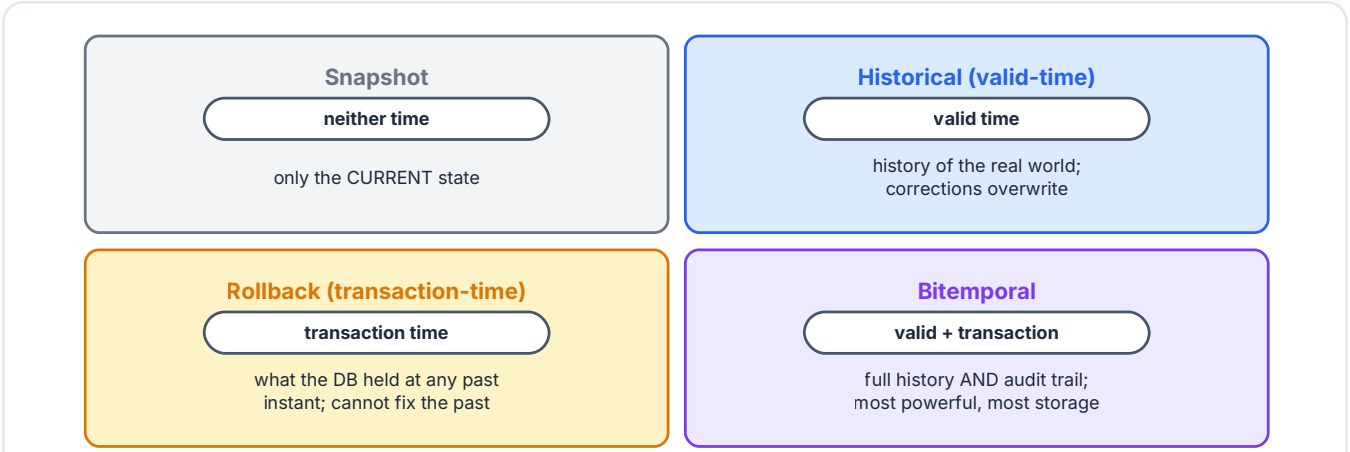


Fig. 62 — Snapshot, historical, rollback and bitemporal databases

Database	Supports
Historical	Valid time only
Rollback	Transaction time only — see the DB exactly as recorded at any past point
Bitemporal	Both — most complete and most complex

🏭 Real-life use case

Financial auditing (regulators ask "what did the books say on 31 March?") · insurance and legal records · medical history · version-controlled data.

📖 GFG: Temporal Databases

5.4 NoSQL Databases

📘 Definition

NoSQL ("Not Only SQL") databases depart from the fixed-schema table model, offering **flexible schemas**, **horizontal scalability** (add machines, not a bigger machine) and handling of large unstructured / semi-structured data.

CAP theorem

📘 Definition

In a distributed data system you can fully guarantee **at most two of three**: **Consistency** (every read sees the latest write), **Availability** (every request gets a non-error response), **Partition tolerance** (works despite lost network links).

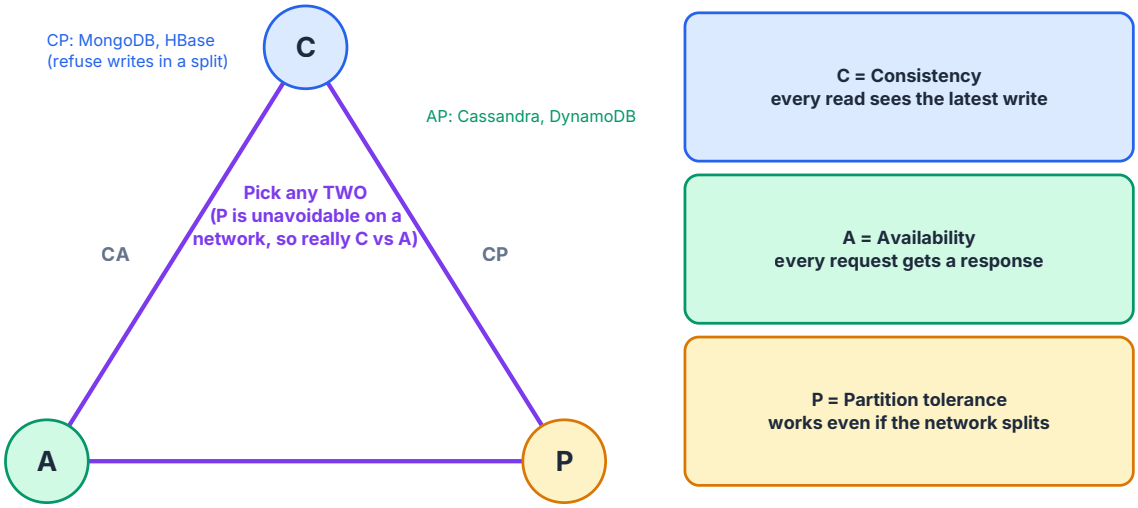


Fig. 63 — CAP triangle — pick two; partition tolerance is not optional

Network partitions are unavoidable, so a distributed DB effectively chooses **CP** (may refuse requests during a partition to avoid stale data) or **AP** (always answers, maybe slightly stale). Most NoSQL systems choose **AP** with **eventual consistency**; relational databases traditionally favour strict **C**.

Four types of NoSQL databases

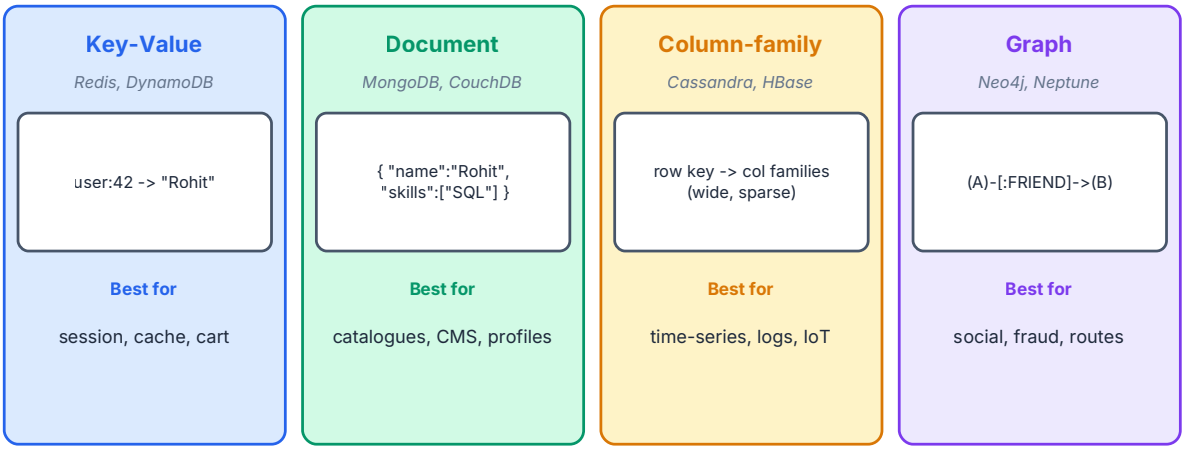


Fig. 64 — Key-value, document, column-family and graph stores

Type	Data model	Examples	Best for
Key-value	key → value; value opaque to DB	Redis, DynamoDB, Riak	Caching, sessions, ultra-fast lookups
Document	Semi-structured JSON / BSON documents; structure may differ	MongoDB, CouchDB	CMS, catalogues, evolving / nested data
Column-family	Column families; rows with varying columns; wide, sparse tables	Cassandra, HBase	Analytics, time-series, logging, write-heavy
Graph	Nodes and edges (§5.1)	Neo4j, ArangoDB	Highly connected data — social, recommendations

SQL vs NoSQL

SQL (relational)	NoSQL
Fixed, predefined schema	Dynamic schema — records can have different fields
Tables of rows and columns	Key-value, documents, wide columns or graphs
Scales **vertically** (bigger server)	Scales **horizontally** (more nodes)
Strong **ACID** consistency	Often **BASE**: availability + eventual consistency
Structured data, complex relationships, transactions	Huge unstructured / semi-structured data, fast writes, horizontal scale
Standard language: SQL	No single standard — MongoDB query language, Cassandra CQL…

ACID vs BASE

ACID (relational)	BASE (many NoSQL systems)
Atomicity, Consistency, Isolation, Durability	**B**asically **A**vailable, **S**oft state, **E**ventual consistency
Correctness first	Availability first
Basically Available — data is always reachable, responsiveness over strict correctness · **Soft state** — state may change over time as replicas converge · **Eventual consistency** — all replicas agree given enough time without new writes.	

Document store example (MongoDB style)

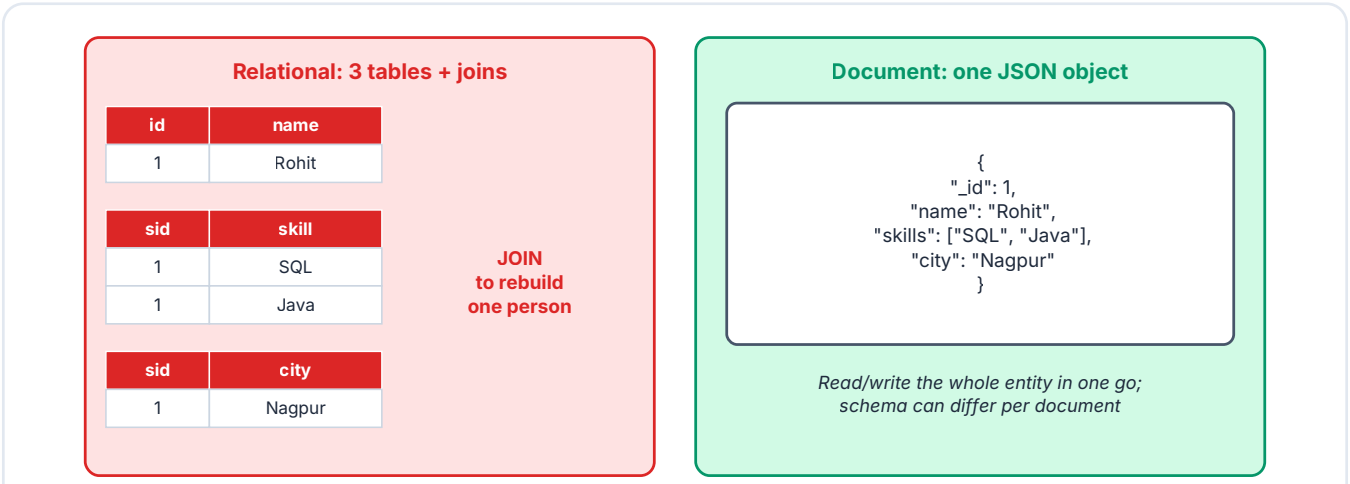


Fig. 65 — The same person: three relational tables vs one JSON document

```json
{
  "_id": "101",
  "name": "Rohit",
  "dept": "CSE",
  "skills": ["Flutter", "Node.js", "PostgreSQL"],
  "address": { "city": "Nagpur", "state": "Maharashtra" }
}
```

In a relational design, multi-valued **skills** need a junction table (1NF) and composite **address** is flattened or split. A document nests arrays and sub-objects directly, matching application JSON and **avoiding joins** for data that is always read together.

🧠 **Yaad rakho**

SQL = ACID, NoSQL = BASE. CAP mein P chhod nahi sakte — to asli choice **C ya A**.

📖 GFG: NoSQL Types · CAP Theorem · SQL vs NoSQL & ACID vs BASE

❓ **Likely exam questions — Unit V**

- What is a graph database? Nodes, edges, index-free adjacency; applications.
- Why is a graph DB better than RDBMS for connected data?
- Spatial data types and query types; why B-tree fails; R-tree / quad-tree.
- Valid time vs transaction time with example; historical / rollback / bitemporal.
- Define NoSQL; CAP theorem and its implication.
- Four types of NoSQL with an example each.
- SQL vs NoSQL; ACID vs BASE.

Quick comparison sheet

One-glance tables for the last day before the exam. Each row is a favourite "differentiate between" question.

Design, relational model and SQL

Topic	Remember
Physical / logical independence	Change internal schema, conceptual unchanged / change conceptual, views unchanged (harder)
Schema / instance	Blueprint, rarely changes / data now, always changes
Super / candidate / primary	Unique set / minimal unique set / chosen candidate
Weak entity	Double rectangle; PK = owner PK + partial key
Specialization / generalization	Top-down / bottom-up
ER → tables	1:M FK on many side; M:N junction table
Degree / cardinality	No. of columns / no. of rows
Algebra / calculus	Procedural (how) / declarative (what); equivalent power
Theta / equi / natural join	Any condition / equality only / equality on same names, one copy of column
WHERE / HAVING	Filters rows before grouping / filters groups after
UNION / UNION ALL	Removes duplicates / keeps them, faster
Correlated subquery	Re-run once per outer row
DELETE / TRUNCATE / DROP	Some rows (DML) / all rows (DDL) / table gone (DDL)
INNER / LEFT / RIGHT / FULL	Overlap / left + overlap / right + overlap / both circles

Normalization and integrity

Topic	Remember
Armstrong's axioms	Reflexivity, Augmentation, Transitivity
Lossless condition	(R1 ∩ R2) → R1 or (R1 ∩ R2) → R2
1NF / 2NF / 3NF	Atomic values / no partial dependency / no transitive dependency
3NF / BCNF	Key *or* prime RHS / determinant must always be a key
4NF / 5NF	No non-trivial MVD / every join dependency implied by keys
Entity / referential integrity	PK never NULL / FK matches a PK or is NULL
Trigger	Event–Condition–Action; BEFORE/AFTER; ROW/STATEMENT
ON DELETE	CASCADE / SET NULL / RESTRICT

Transactions, recovery and modern databases

Topic	Remember
ACID	All-or-nothing, valid state, as if alone, permanent
Txn states	Active → Partially committed → Committed / Failed → Aborted
Concurrency problems	Lost update, dirty read, incorrect summary, unrepeatable read
Conflict serializable	Precedence graph has no cycle
Concurrency control	Lock (2PL), timestamp, optimistic validation, MVCC
Deadlock	Prevention (Wait-Die / Wound-Wait) vs detection (wait-for graph cycle) + victim
Deferred / immediate	REDO only / UNDO + REDO
Checkpoint	Recovery starts from the last checkpoint, not log start
Query processing	Parse → optimise (cheapest plan) → evaluate
Valid / transaction time	True in real world / stored in database
CAP	Pick two of C, A, P — P is mandatory, so C vs A
NoSQL types	Key-value, document, column-family, graph
ACID / BASE	Strict consistency / basically available, soft state, eventual consistency

Exam-day revision checklist

If you can **write or draw** each of these from memory, you cover the highest-weightage recurring questions.

- [ ] Advantages of DBMS over file-processing (all 7)
- [ ] Three-schema diagram + physical vs logical independence
- [ ] Keys: super, candidate, primary, alternate, foreign
- [ ] Weak vs strong entity sets + example
- [ ] Specialization, generalization, aggregation, inheritance
- [ ] ER → tables rules (all 6 cases)
- [ ] Relational algebra operators with symbols and examples
- [ ] Relational algebra vs relational calculus table
- [ ] WHERE vs HAVING; correlated vs non-correlated subquery
- [ ] SQL JOIN types with Venn diagrams
- [ ] DELETE vs TRUNCATE vs DROP
- [ ] Armstrong's axioms (R-A-T)
- [ ] Insertion / deletion / update anomalies with example
- [ ] 1NF → 5NF with condition for each
- [ ] 3NF vs BCNF comparison
- [ ] Entity vs referential integrity
- [ ] Triggers: event-condition-action, BEFORE/AFTER, ROW/STATEMENT
- [ ] ACID with one example each
- [ ] Transaction state diagram
- [ ] Lost update, dirty read, incorrect summary, unrepeatable read
- [ ] Conflict serializability by precedence graph (practise one numerical)
- [ ] Deadlock: prevention vs detection & recovery
- [ ] Deferred vs immediate modification (UNDO / REDO)
- [ ] Checkpoints — why needed
- [ ] Query processing: parse → optimise → evaluate
- [ ] CAP theorem + why P is non-negotiable
- [ ] 4 NoSQL types with one example DB each
- [ ] SQL vs NoSQL table; ACID vs BASE
- [ ] Valid time vs transaction time
- [ ] Graph DB: nodes, edges, index-free adjacency
- [ ] Spatial DB: R-tree / quad-tree, query types

Recommended resources

Resource	Use it for
GeeksforGeeks — DBMS	Topic-wise articles and practice questions
PostgreSQL documentation	Try SQL, views, triggers, transactions, EXPLAIN, PostGIS
Neo4j Cypher manual	Graph queries
MongoDB manual	Document store examples
SQLite · SQLite Online	Practise SQL in the browser with zero setup

Diagrams in this book are original drawings made for these notes; use them as a model for the figures you draw in the exam.